

APOSTILA - AULA 04

COMPLEXIDADE DE ALGORITMOS

Contagem de operações e as notações O , Ω e Θ

com exemplos, exercícios resolvidos e a atividade prática Campo Minado



Prof. Filipino Novo Mór

filipomor.com

Material de apoio da Aula 4 (Desempenho de Algoritmos), para estudar depois da aula ao vivo ou junto com a gravação. Os códigos citados estão no repositório <https://github.com/ProfessorFilipo/EstruturasDados>, na pasta aula04-complexidade.

Sumário

Como usar esta apostila	2
1 Por que analisar algoritmos?	4
1.1 O mesmo problema, duas soluções	4
1.2 Por que o cronômetro engana	5
1.3 A ideia central: contar passos	6
1.4 O caminho desta apostila	6
2 Contando operações	8
2.1 O modelo de custo	8
2.2 Exemplo 1: somar um vetor	8
2.3 Exemplo 2: busca linear	10
2.4 Exemplo 3: laços aninhados	10
2.5 Exemplo 4: um laço que divide por 2	11
2.6 Cuidado com o Python: uma linha não é um passo	11
3 Matemática de bolso	13
3.1 Funções: uma máquina de entrada e saída	13
3.2 Funções crescentes	14
3.3 Potências e raízes	15
3.4 Logaritmos: quantas vezes dá para dividir ao meio?	16
3.5 Exponenciais: dobrar a cada passo	17
3.6 Fatorial: todas as ordens possíveis	19
3.7 Somatórios e a soma de Gauss	20
3.8 Para n muito grande	21
4 Termo dominante e análise assintótica	23
4.1 Quem manda quando n cresce	23
4.2 As duas regras de simplificação	24
4.3 Análise assintótica	24



4.4	Um cuidado: desprezar não é ignorar sempre	25
5	Notação O: o teto	26
5.1	A ideia	26
5.2	A definição, palavra por palavra	26
5.3	Exemplo guiado: $3n + 4 = O(n)$	27
5.4	Uma técnica que sempre funciona para polinômios	27
5.5	Como mostrar que algo NÃO é O	28
5.6	O é um teto — e pode ser folgado	28
6	Notação Ω: o piso	30
6.1	A definição	30
6.2	Exemplos	31
6.3	Ω também descreve problemas	31
6.4	Como mostrar que algo NÃO é Ω	31
7	Notação Θ: o crescimento exato	32
7.1	A definição	32
7.2	Exemplos	33
7.3	Resumo das três notações	33
7.4	Uma regra prática para polinômios	33
8	Melhor, pior e caso médio	35
8.1	Três perguntas diferentes	35
8.2	A busca linear, caso a caso	35
8.3	Qual caso importa mais?	36
8.4	Mais dois exemplos das aulas anteriores	36
8.5	Casos e notações não são a mesma coisa	37
9	Estudo de caso: Insertion Sort	38
9.1	A ideia: arrumar cartas na mão	38
9.2	O código	38
9.3	Passo a passo	39
9.4	Melhor caso: o vetor já está ordenado	39
9.5	Pior caso: ordem inversa	40
9.6	Caso médio: ordem aleatória	40
9.7	Vendo na prática	40



9.8	Onde entram O , Ω e Θ	41
9.9	Dados quase ordenados	42
10	Comparando famílias de funções	43
10.1	A hierarquia	43
10.2	Os números	44
10.3	Lendo um gráfico em escala logarítmica	44
10.4	Transformando em tempo	45
10.5	E um computador mil vezes mais rápido?	45
11	Um toque de Cálculo	47
11.1	Derivada: a inclinação de uma curva	47
11.2	A regra da potência	48
11.3	Custo marginal: quanto custa um elemento a mais?	48
11.4	Achando o melhor parâmetro: a busca por saltos	49
11.5	Comparando crescimentos com limites	50
12	Regras práticas para analisar código	52
12.1	Os padrões	52
12.2	Exemplos resolvidos	52
13	Revisitando Lista, Pilha, Fila e Conjunto	55
13.1	A reflexão da Aula 2, respondida	56
13.2	Escolhendo a estrutura	56
14	Campo Minado: regras do jogo e técnicas de implementação	57
14.1	As regras do jogo	57
14.2	A organização do programa	58
14.3	Técnica 1: o tabuleiro numa lista linear	58
14.4	Técnica 2: vizinhança e bordas	60
14.5	Técnica 3: sortear minas sem repetição	60
14.6	Técnica 4: calcular os números percorrendo as minas	61
14.7	Técnica 5: a abertura em cascata com Fila ou Pilha	61
14.8	Técnica 6: vitória em $O(1)$	63
14.9	Técnica 7: histórico com Pilha e o desfazer	63
14.10	Técnica 8: ranking com inserção ordenada	63
14.11	Técnica 9: medir contando operações	63



14.12	Técnica 10: interfaces, eventos e testes	64
14.13	Resumo dos custos	64
15	Lista de exercícios resolvidos	65
15.1	Bloco A — contagem e matemática	65
15.2	Bloco B — notações	66
15.3	Bloco C — casos e algoritmos	67
15.4	Bloco D — questões objetivas (estilo da prova)	69
15.5	Bloco E — programação	71
16	Resumo para a prova	72
16.1	As ideias centrais	72
16.2	Fórmulas de bolso	72
16.3	As notações	73
16.4	A hierarquia e as regras práticas	73
16.5	Casos que caem na prova	73
16.6	Glossário	74
16.7	Símbolos e como ler	74
A	Mapa: slides da aula e apostila	75

Como usar esta apostila

Esta apostila acompanha a Aula 4 de Estruturas de Dados, a última do Grau 1. Na aula ao vivo, o tempo é curto; aqui você encontra cada ideia com mais calma, com mais exemplos e com os cálculos feitos passo a passo. Ela foi escrita pensando em quem **não teve uma base forte de matemática**: tudo o que é preciso — funções, potências, logaritmos, somatórios e até uma pitada de Cálculo — é explicado do zero, sempre ligado ao que você já programou nas Aulas 1 a 3.

Com os slides e a gravação

A apostila segue a mesma ordem da aula. Logo abaixo do título de cada capítulo aparece a indicação **NA AULA: slides ...**, com os slides que tratam daquele assunto. E cada slide da aula traz, no canto, a página da apostila que aprofunda o tema. Se estiver assistindo à gravação, pause o vídeo quando quiser e leia o trecho correspondente.

As caixas que você vai encontrar

INTUIÇÃO Uma explicação informal, com analogias, antes da definição exata. Leia com calma: é aqui que a ideia “cai a ficha”.

CONEXÃO COM AS AULAS ANTERIORES Uma ligação com o que você já estudou nas Aulas 1 a 3: listas, pilhas, filas e conjuntos.

ATENÇÃO Um erro comum, um detalhe que costuma confundir ou um ponto que aparece muito em provas.

EXERCÍCIO RÁPIDO Um exercício curto para você fazer na hora, com lápis e papel. As respostas estão no fim de cada capítulo — tente antes de olhar!

APROFUNDE (OPCIONAL) Um assunto um pouco mais avançado. Pode pular na primeira leitura sem prejuízo.



Como estudar

1. Leia um capítulo por vez, com lápis e papel ao lado. Refaça as contas dos exemplos.
2. Faça cada **exercício rápido** antes de ver a resposta.
3. Rode os programas citados (pasta aula04 - complexidade do repositório). Eles funcionam na IDE online usada nas aulas.
4. No fim, resolva a lista de exercícios do capítulo 15 e use o resumo do capítulo 16 para revisar antes da prova do Grau 1.
5. Para praticar tudo junto, faça a atividade Campo Minado (capítulo 14 e enunciado próprio).

01

Por que analisar algoritmos?

NA AULA: slides 04 e 05 | NO REPOSITÓRIO: aula04-complexidade/

Desde a Aula 1, usamos expressões como “acesso por índice é $O(1)$ ” e “inserir no início da lista contígua é $O(n)$ ”. Neste capítulo você vai entender *por que* precisamos desse tipo de medida e por que ela não é dada em segundos.

CONEXÃO COM AS AULAS ANTERIORES Esta é a tabela que abriu o nosso curso, na Aula 1. Ao final desta apostila, você vai saber explicar cada célula dela — e calcular tabelas como essa sozinho.

OPERAÇÃO	CONTÍGUA (ARRAY)	ENCADEADA (LISTA)
Acesso por índice	$O(1)$	$O(n)$
Inserir/remover no início	$O(n)$	$O(1)$
Inserir/remover no fim	$O(1)$	$O(1)$

1.1. O mesmo problema, duas soluções

Considere um problema simples: somar todos os números de 1 até n . Eis duas soluções corretas, que sempre dão a mesma resposta:



```
def soma_com_laco(n):
    total = 0
    for i in range(1, n + 1): # repete n vezes
        total = total + i
    return total

def soma_com_formula(n):
    return n * (n + 1) // 2 # sempre 3 contas: *, + e //
```

Para $n = 10$, as duas devolvem 55 num piscar de olhos. Mas repare no trabalho de cada uma: `soma_com_laco` faz uma soma para cada número — para n igual a um bilhão, um bilhão de somas. Já `soma_com_formula` faz sempre as mesmas 3 contas, seja n igual a 10 ou a um bilhão. (A fórmula vem de um truque famoso, a *soma de Gauss*, que você verá na seção 3.7.)

INTUIÇÃO Dois algoritmos podem resolver o mesmo problema com esforços completamente diferentes. Analisar um algoritmo é responder: **como o trabalho cresce quando a entrada cresce?** No primeiro, o trabalho cresce junto com n ; no segundo, fica sempre igual.

1.2. Por que o cronômetro engana

A primeira ideia para comparar algoritmos é medir o tempo. Foi o que fizemos no exercício 14 da Aula 1. O problema é que o tempo depende de muitas coisas além do algoritmo: o processador, a linguagem, outros programas abertos, até a temperatura da máquina. O mesmo código leva tempos diferentes em computadores diferentes — e até em duas execuções no mesmo computador.

Veja uma medição real, feita no computador onde esta apostila foi gerada: o mesmo algoritmo de ordenação (o Insertion Sort, que estudaremos no capítulo 9), com o vetor em ordem inversa, escrito em C e em Python.

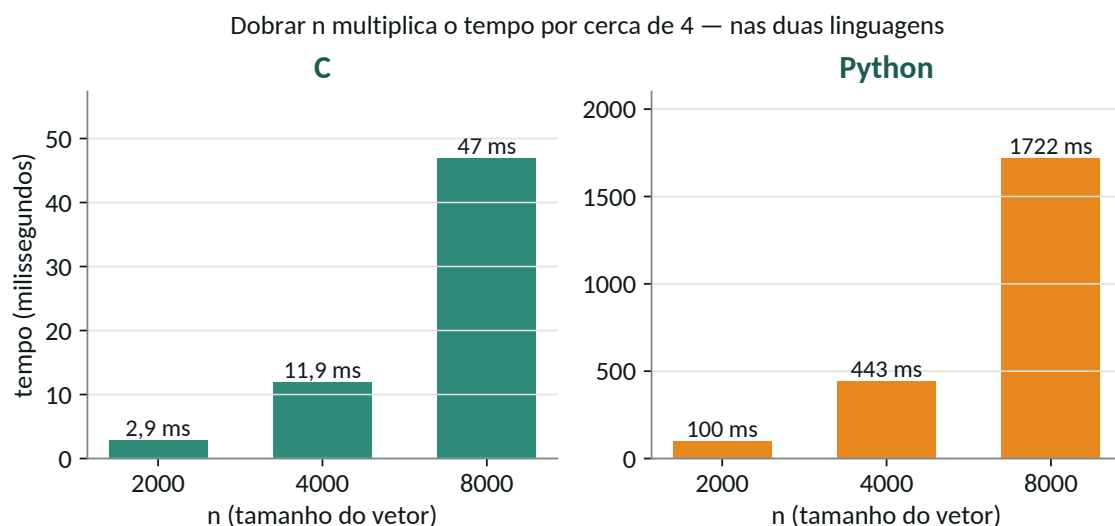


Figura 1.1. Tempo do mesmo algoritmo em C e em Python, para vetores de 2.000, 4.000 e 8.000 elementos. Repare nas escalas: o Python é cerca de 35 vezes mais lento.



n	2.000	4.000	8.000
Tempo em C	2,9 ms	11,9 ms	47,0 ms
Tempo em Python	100 ms	443 ms	1.722 ms
Quanto multiplicou ao dobrar n	—	$\approx 4\times$	$\approx 4\times$

Os números absolutos são muito diferentes: 47 milissegundos contra quase 2 segundos. Mas o **comportamento** é o mesmo nas duas linguagens: cada vez que n dobra, o tempo fica cerca de 4 vezes maior. Essa é a informação que vale em qualquer máquina e em qualquer linguagem — e é ela que a análise de algoritmos captura.

ATENÇÃO Os tempos da tabela vão mudar no seu computador. O “multiplicou por 4” não vai. É por isso que, em vez de segundos, vamos medir algoritmos pelo **número de passos** e, mais ainda, pela **forma como esse número cresce**.

1.3. A ideia central: contar passos

Em vez de cronometrar, vamos **contar** quantas operações simples o algoritmo executa em função do tamanho da entrada, que chamaremos de n . O resultado é uma função, que escrevemos como $T(n)$ — lê-se “tê de ene”. Por exemplo:

- `soma_com_laco` faz um número de passos proporcional a n ;
- `soma_com_formula` faz sempre o mesmo número de passos, não importa n .

Contar passos não depende da máquina: um computador mais rápido executa cada passo mais depressa, mas a quantidade de passos é a mesma. No capítulo 2 você vai aprender a fazer essa contagem com precisão, e nos capítulos seguintes, a resumir o resultado com as notações O , Ω e Θ .

1.4. O caminho desta apostila

CAPÍTULOS	O QUE VOCÊ VAI APRENDER
2	contar os passos de um algoritmo e escrever $T(n)$
3 e 4	a matemática necessária, do zero: funções, potências, logaritmos, somatórios, termo dominante
5, 6 e 7	as notações O (teto), Ω (piso) e Θ (justo)
8 e 9	melhor, pior e caso médio — com o Insertion Sort como estudo de caso
10 e 11	comparar famílias de funções e usar derivadas para entender o crescimento
12 e 13	regras práticas para analisar código e a revisão das Aulas 1 a 3
14	o Campo Minado: regras, técnicas e análise de custo
15 e 16	lista de exercícios resolvidos e resumo para a prova



EXERCÍCIO RÁPIDO 1.1 Para $n = 1.000.000$, quantas somas `soma_com_laco` faz? E quantas contas faz `soma_com_formula`? Se cada conta levasse 1 microssegundo, quanto tempo cada uma levaria?

Respostas dos exercícios rápidos

1.1. `soma_com_laco` faz 1.000.000 de somas (uma por número), ou seja, cerca de 1 segundo a 1 microssegundo por soma. `soma_com_formula` faz sempre 3 contas: cerca de 3 microssegundos. E se n fosse um bilhão, a primeira levaria uns 17 minutos; a segunda, os mesmos 3 microssegundos.



Contando operações

NA AULA: slides 06 a 08 | NO REPOSITÓRIO: aula04-complexidade/

Neste capítulo vamos transformar um trecho de código em uma função $T(n)$: o número de passos que ele executa para uma entrada de tamanho n .

2.1. O modelo de custo

Precisamos combinar o que conta como “um passo”. Usaremos um modelo simples, que é o mesmo dos slides e dos programas da aula:

CUSTA 1 PASSO CADA VEZ QUE EXECUTA	EXEMPLO
uma atribuição	<code>total = 0</code>
um teste de laço ou de <code>if</code>	<code>i < n, v[i] == valor</code>
um incremento	<code>i++, i = i + 1</code>
um comando simples do corpo	<code>total = total + v[i]</code>
um <code>return</code>	<code>return total</code>

INTUIÇÃO Na vida real, `total = total + v[i]` envolve várias operações da máquina (ler `v[i]`, somar, guardar). Contar isso como 1 passo parece grosseiro, mas não faz diferença no que nos interessa: se cada “passo” na verdade custar 3 operações, o total só fica multiplicado por 3 — e, como você verá no capítulo 4, constantes multiplicativas não mudam a forma como o custo cresce.

O segredo da contagem é perguntar, para cada linha: **quantas vezes ela executa?** Multiplicamos o custo da linha por esse número e somamos tudo.

2.2. Exemplo 1: somar um vetor



```
int soma(const int v[], int n) {
    int total = 0;           /* custo 1, executa 1 vez          */
    for (int i = 0; i < n; i++) { /* i = 0: 1 vez | i < n: n + 1 vezes | i++: n vezes */
        total = total + v[i]; /* custo 1, executa n vezes          */
    }
    return total;           /* custo 1, executa 1 vez          */
}
```

LINHA	VEZES	PASSOS
<code>int total = 0;</code>	1	1
<code>int i = 0</code>	1	1
<code>i < n</code>	$n + 1$	$n + 1$
<code>total = total + v[i];</code>	n	n
<code>i++</code>	n	n
<code>return total;</code>	1	1
Total		$3n + 4$

Por que o teste `i < n` executa $n + 1$ vezes, e não n ? Porque ele é avaliado uma vez para cada volta do laço (dando verdadeiro) e **mais uma vez no final**, quando dá falso e o laço termina. Esse “mais um” é o detalhe que mais gente esquece.

n	$T(n) = 3n + 4$
10	34
100	304
1.000	3.004

Em Python, o mesmo cálculo pode ser escrito com `while`, deixando cada passo visível. A contagem é idêntica:

```
def soma(v, n):
    total = 0          # 1 vez
    i = 0              # 1 vez
    while i < n:        # n + 1 vezes
        total = total + v[i] # n vezes
        i = i + 1          # n vezes
    return total        # 1 vez
```

DICA O programa `python/contagem_operacoes.py` (e a versão em C) conta os passos de verdade, com um contador, e compara com a fórmula $3n + 4$. Rode e confira que os números batem.



2.3. Exemplo 2: busca linear

Esta é a mesma lógica do `lista_buscar()` da Aula 1: procurar um valor olhando as posições uma a uma.

```
int buscar(const int dados[], int tamanho, int valor) {
    for (int i = 0; i < tamanho; i++) {
        if (dados[i] == valor) {
            return i;
        }
    }
    return -1;
}
```

Agora aparece uma novidade: o número de passos **depende dos dados**, e não só de n .

- **Valor ausente (pior situação):** o laço dá todas as voltas. São 1 passo para $i = 0$, $n + 1$ testes $i < n$, n comparações no `if`, n incrementos e 1 `return -1`: $T(n) = 3n + 3$. Para $n = 100$, 303 passos.
- **Valor na posição 0 (melhor situação):** $i = 0$, um teste, uma comparação e o `return`: $T(n) = 4$, qualquer que seja n .

Essa diferença entre “melhor” e “pior” situação é tão importante que ganha um capítulo inteiro, o 8.

2.4. Exemplo 3: laços aninhados

O que acontece com um laço dentro de outro? Este trecho imprime todos os pares (i, j) :

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        printf("%d %d\n", i, j);
    }
}
```

O laço de fora faz o de sempre: 1 inicialização, $n + 1$ testes e n incrementos. O de dentro **roda inteiro a cada volta** do de fora, ou seja, n vezes:

LINHA	VEZES
<code>int i = 0; i < n; i++</code>	$1 + (n + 1) + n$
<code>int j = 0</code> (uma vez por volta de fora)	n
<code>j < n</code> ($n + 1$ por volta de fora)	$n(n + 1) = n^2 + n$
<code>j++</code> e <code>printf</code> (n cada, por volta de fora)	$n^2 + n^2$
Total	$3n^2 + 4n + 2$

Um laço dentro de outro **multiplica**: n voltas de fora $\times$ n voltas de dentro dão n^2 execuções do corpo. Guarde essa ideia; ela volta no capítulo 12.



2.5. Exemplo 4: um laço que divide por 2

```
i = n
while i > 1:
    i = i // 2      # divisão inteira por 2
```

Com $n = 16$, i vale 16, 8, 4, 2 e 1: o corpo executa 4 vezes, e o teste $i > 1$, 5 vezes. Total: $1 + 5 + 4 = 10$ passos. Com $n = 1.024$, seriam só 10 divisões. Esse “quantas vezes dá para dividir ao meio” tem nome — é o **logaritmo**, assunto da seção 3.4. O custo deste laço é $T(n) = 2 \log_2 n + 2$.

2.6. Cuidado com o Python: uma linha não é um passo

Linguagens como Python escondem laços dentro de operações que parecem simples. No nosso modelo, uma linha só vale 1 passo se ela realmente fizer uma quantidade fixa de trabalho.

OPERAÇÃO EM PYTHON	CUSTO	POR QUÊ
<code>v[i], len(v)</code>	$O(1)$	lista contígua: acesso direto (Aula 1)
<code>v.append(x)</code>	$O(1)^*$	escreve no fim; às vezes realoca (Aula 1)
<code>x in v</code>	$O(n)$	percorre a lista procurando x
<code>v.insert(0, x)</code>	$O(n)$	desloca todos os elementos (Aula 1)
<code>v.remove(x), v.index(x)</code>	$O(n)$	procura e depois desloca
<code>min(v), max(v), sum(v)</code>	$O(n)$	olham todos os elementos
<code>v.pop()</code>	$O(1)$	remove do fim, como o desempilhar (Aula 2)

* Em média. De vez em quando, a lista precisa de mais espaço e é copiada inteira, mas isso é raro o suficiente para, na média, cada `append` custar um número fixo de passos.

ATENÇÃO `if valor in lista:` parece um único teste, mas é uma busca linear inteira! Foi por isso que, na Aula 3, o `pertence()` do nosso Conjunto era $O(n)$. Se essa linha estiver dentro de um laço de n voltas, o total vira $n \times n = n^2$.



EXERCÍCIO RÁPIDO 2.1 Conte os passos da função abaixo, que devolve o maior valor de um vetor com $n \geq 1$ elementos. (a) Quantos passos, se o maior valor estiver na posição 0? (b) E se o vetor estiver em ordem crescente, fazendo $m = v[i]$ executar em todas as voltas?

```
int maior(const int v[], int n) {  
    int m = v[0];  
    for (int i = 1; i < n; i++) {  
        if (v[i] > m) {  
            m = v[i];  
        }  
    }  
    return m;  
}
```

EXERCÍCIO RÁPIDO 2.2 Na busca linear com o valor ausente e $n = 50$: (a) quantas vezes o `if` é avaliado? (b) quantos passos ao todo?

Respostas dos exercícios rápidos

2.1. Repare que o laço começa em $i = 1$: ele dá $n - 1$ voltas e o teste $i < n$ executa n vezes. Contando: $m = v[0]$ (1) + $i = 1$ (1) + testes (n) + `if` ($n - 1$) + incrementos ($n - 1$) + `return` (1). (a) Sem nenhuma troca de m : $1 + 1 + n + (n - 1) + (n - 1) + 1 = 3n + 1$. (b) Com $m = v[i]$ executando em todas as $n - 1$ voltas: $3n + 1 + (n - 1) = 4n$. Nos dois casos, o custo cresce proporcionalmente a n .

2.2. (a) 50 vezes, uma por posição. (b) $3 \cdot 50 + 3 = 153$ passos.



Matemática de bolso

NA AULA: slides 09 a 16 | NO REPOSITÓRIO: aula04-complexidade/

Este capítulo reúne toda a matemática de que você vai precisar, explicada do zero. Não pule nenhuma seção, mesmo que o assunto pareça conhecido: cada uma termina ligando a ideia a algo que você já programou.

3.1. Funções: uma máquina de entrada e saída

Uma **função** é uma regra que recebe um valor de entrada e devolve um valor de saída. Pense numa máquina: você coloca um número de um lado e sai um número do outro.

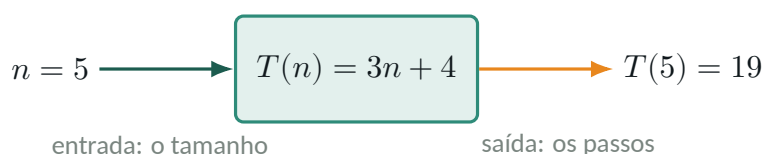


Figura 3.1. A função $T(n) = 3n + 4$, que conta os passos da soma de um vetor (capítulo 2).

Escrevemos $T(n)$ para dizer “o valor de T quando a entrada é n ”. Para calcular, basta trocar n pelo número: $T(5) = 3 \cdot 5 + 4 = 19$. Em análise de algoritmos, a entrada n é quase sempre o **tamanho** dos dados — o mesmo número que o `tamanho()` da nossa Lista devolve — e a saída é o **custo**: passos, comparações, deslocamentos.

3.1.1 Tabela e gráfico

Uma função pode ser mostrada como tabela ou como gráfico. No gráfico, cada ponto tem duas coordenadas: a posição na horizontal é a entrada n ; a altura é a saída $T(n)$.

n	0	1	2	5	8	10
$T(n) = 3n + 4$	4	7	10	19	28	34

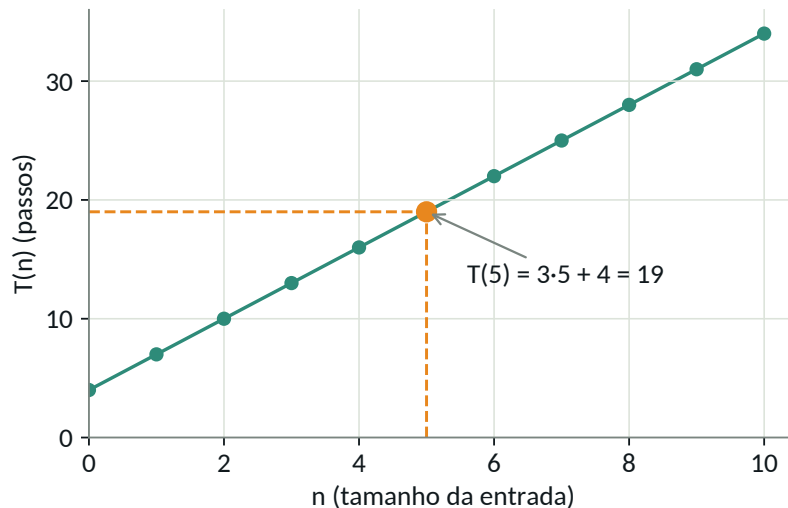


Figura 3.2. Como ler um gráfico: ande na horizontal até $n = 5$, suba até a linha e olhe a altura à esquerda: 19.

3.2. Funções crescentes

Uma função é **crescente** quando, ao aumentar a entrada, a saída nunca diminui. Em símbolos: se $a < b$, então $f(a) \leq f(b)$. O custo de um algoritmo é (quase sempre) uma função crescente: mais dados, mais trabalho. O que muda de um algoritmo para outro é a **velocidade** com que o custo cresce.

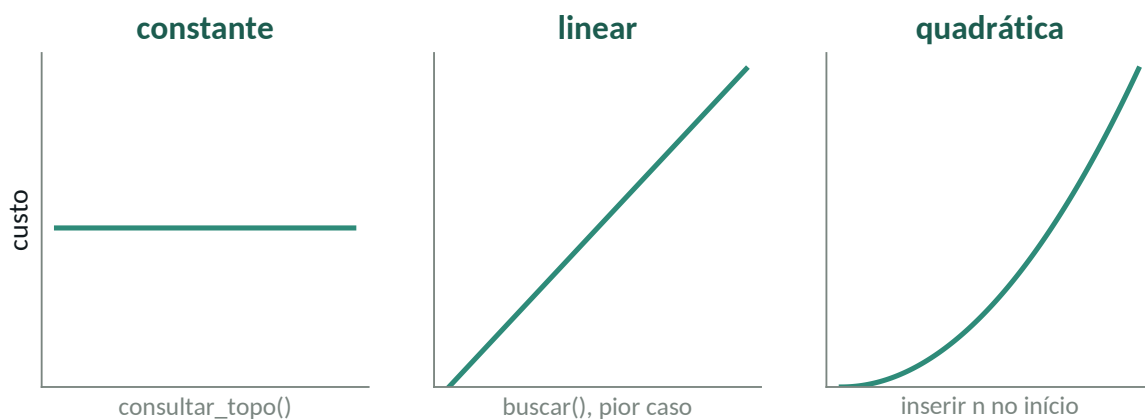


Figura 3.3. Três formas de crescer, com exemplos das Aulas 1 e 2.

- **Constante:** o custo não depende de n . O `consultar_topo()` da Pilha custa o mesmo com 3 ou com 3 milhões de elementos. (Uma função constante conta como crescente no sentido de que “nunca diminui”.)
- **Linear:** o custo cresce na mesma proporção de n . Dobrou n , dobrou o custo. É o caso da busca linear quando o valor não está na lista.
- **Quadrática:** o custo cresce muito mais rápido. Dobrou n , o custo quadruplica. É o caso de inserir n elementos, um a um, no início de uma lista contígua.



EXERCÍCIO RÁPIDO 3.1 Considere $f(n) = 7$, $g(n) = 2n + 1$ e $h(n) = n^2$. (a) Calcule as três para $n = 1, 5$ e 10 . (b) Em cada caso, qual é a maior? (c) O que isso sugere sobre comparar funções olhando só valores pequenos de n ?

3.3. Potências e raízes

A **potência** n^k é n multiplicado por ele mesmo k vezes: $n^2 = n \cdot n$ (lê-se “ene ao quadrado”) e $n^3 = n \cdot n \cdot n$ (“ene ao cubo”). Os nomes vêm da geometria: n^2 é a área de um quadrado de lado n ; n^3 , o volume de um cubo.

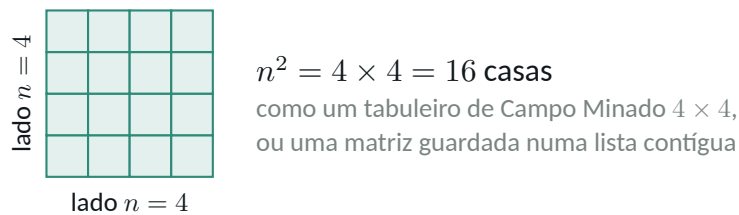


Figura 3.4. n^2 conta as casas de um quadrado de lado n .

A **raiz quadrada** faz o caminho inverso: $\sqrt{n}$ é o lado de um quadrado com n casas. $\sqrt{16} = 4$, $\sqrt{100} = 10$, $\sqrt{10.000} = 100$. Também se escreve $\sqrt{n} = n^{1/2}$.

3.3.1 Regras que vamos usar

REGRA	EXEMPLO	EM PALAVRAS
$n^a \cdot n^b = n^{a+b}$	$2^3 \cdot 2^4 = 2^7 = 128$	multiplicar: soma os expoentes
$(n^a)^b = n^{a \cdot b}$	$(n^2)^3 = n^6$	potência de potência: multiplica
$n^1 = n$ e $n^0 = 1$	$7^0 = 1$	qualquer número (não nulo) elevado a 0 dá 1
$n^{-1} = \frac{1}{n}$	$10^{-2} = \frac{1}{100}$	expoente negativo: inverte
$n^{1/2} = \sqrt{n}$	$64^{1/2} = 8$	expoente $\frac{1}{2}$ é a raiz quadrada

CONEXÃO COM AS AULAS ANTERIORES Na Aula 3, a interseção “ingênua” de dois Conjuntos com n e m elementos perguntava, para cada elemento do primeiro, se ele pertencia ao segundo: n consultas de custo m , ou seja, $n \cdot m$ comparações. Se os dois têm o mesmo tamanho, isso dá n^2 .

EXERCÍCIO RÁPIDO 3.2 Calcule: (a) $2^3 \cdot 2^4$; (b) $\sqrt{64}$; (c) $(n^2)^3$; (d) 10^{-2} ; (e) $1.000^{1/3}$ (dica: que número multiplicado por ele mesmo 3 vezes dá 1.000?).



3.4. Logaritmos: quantas vezes dá para dividir ao meio?

Pegue 16 caixas e vá dividindo ao meio até sobrar uma. Quantas divisões são necessárias?

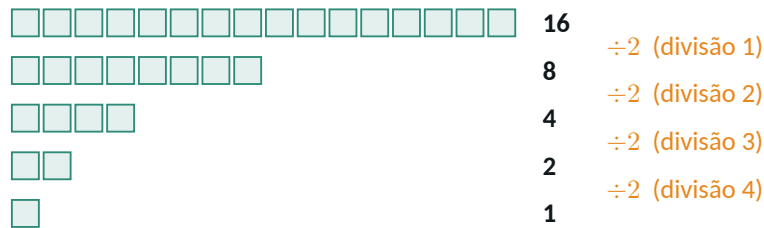


Figura 3.5. $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: são 4 divisões. Por isso, $\log_2 16 = 4$.

O número de divisões ao meio até chegar a 1 é o **logaritmo na base 2**. Escrevemos $\log_2 16 = 4$ (lê-se “logaritmo de 16 na base 2 é 4”). Olhando ao contrário: dobrando 4 vezes a partir de 1, chegamos a 16, ou seja, $2^4 = 16$. Esta é a definição:

$$\log_2 n = x \iff 2^x = n$$

n	1	2	8	16	1.024	1.048.576	≈ 1 BILHÃO
$\log_2 n$	0	1	3	4	10	20	≈ 30

Repare no mais importante: quando n vai de mil para um milhão (mil vezes mais), o logaritmo só vai de 10 para 20. **O logaritmo cresce muito devagar.** Um algoritmo que faz $\log_2 n$ passos é excelente.

INTUIÇÃO O jogo “adivinha o número de 1 a 100”: a cada palpite, você ouve “maior” ou “menor”. Chutando sempre o meio, cada resposta elimina metade dos números que sobram. Em no máximo 7 palpites você acerta, porque $2^7 = 128 \geq 100$. Esse é o princípio da *busca binária* em um vetor ordenado: cerca de $\log_2 n$ comparações.

3.4.1 E quando n não é uma potência de 2?

O logaritmo continua existindo, só que não é inteiro: $\log_2 1.000 \approx 9,97$, porque $2^{9,97} \approx 1.000$. Para calcular numa calculadora (ou no Python), use:

$$\log_2 n = \frac{\ln n}{\ln 2} \approx 3,32 \cdot \log_{10} n \quad \text{no Python: } \text{math.log2}(n)$$

3.4.2 Propriedades úteis

PROPRIEDADE	EXEMPLO	EM PALAVRAS
$\log(a \cdot b) = \log a + \log b$	$\log_2(8 \cdot 4) = 3 + 2 = 5$	produto vira soma
$\log(n^k) = k \cdot \log n$	$\log_2(2^{10}) = 10$	o expoente “desce”
$\log 1 = 0$	$\log_2 1 = 0$	porque $2^0 = 1$
$\log_2 n = 3,32 \cdot \log_{10} n$	$\log_2 1.000 \approx 3,32 \cdot 3$	bases diferentes só diferem por uma constante



ATENÇÃO A última propriedade tem uma consequência importante: em análise de algoritmos, a base do logaritmo **não importa**. Trocar de base só multiplica o resultado por uma constante — e constantes são desprezadas (capítulo 4). Por isso escrevemos simplesmente $O(\log n)$.

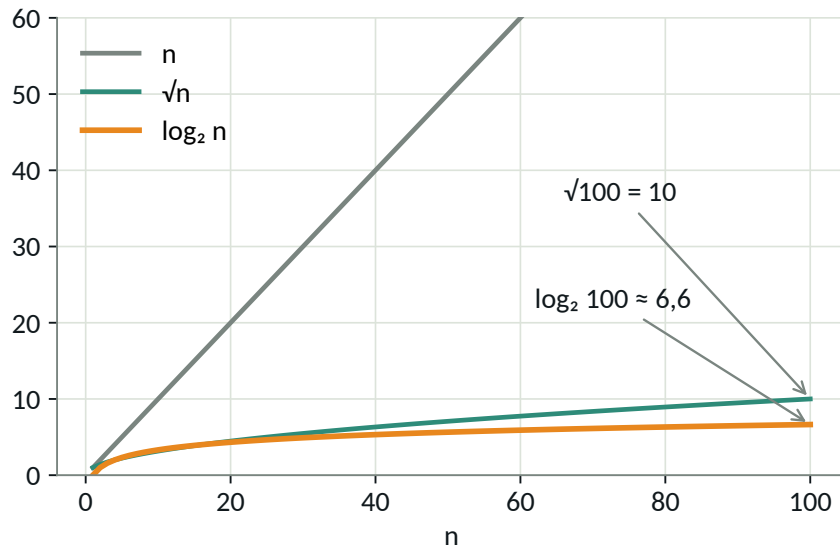


Figura 3.6. Para $n = 100$: n vale 100, $\sqrt{n}$ vale 10 e $\log_2 n$ vale só 6,6.

EXERCÍCIO RÁPIDO 3.3 (a) $\log_2 64$; (b) $\log_2 1.024$; (c) entre quais inteiros está $\log_2 100$? (d) Quantas vezes é preciso dividir 1.000.000 ao meio para chegar a 1 (aproximadamente)?

3.5. Exponenciais: dobrar a cada passo

Na **exponencial** 2^n , a variável está no expoente: cada vez que n aumenta 1, o valor **dobra**. $2^{10} = 1.024$, $2^{20} \approx$ um milhão, $2^{30} \approx$ um bilhão.

INTUIÇÃO Diz a lenda que o inventor do xadrez pediu como prêmio 1 grão de trigo na primeira casa do tabuleiro, 2 na segunda, 4 na terceira, dobrando até a casa 64. Parece pouco, mas a última casa sozinha teria 2^{63} grãos, e o total, $2^{64} - 1 \approx 1,8 \times 10^{19}$ grãos — muito mais trigo do que o mundo inteiro já produziu.

Onde a exponencial aparece em computação? Sempre que um algoritmo testa **todas as combinações** de escolhas do tipo sim/não.

CONEXÃO COM AS AULAS ANTERIORES Um Conjunto (Aula 3) com n elementos tem 2^n subconjuntos. Para cada elemento, há duas escolhas: ele está no subconjunto ou não. São $2 \times 2 \times \dots \times 2 = 2^n$ combinações. Para $\{A, B, C\}$:

A ESTÁ?	B ESTÁ?	C ESTÁ?	SUBCONJUNTO
não	não	não	$\emptyset$ (vazio)
não	não	sim	$\{C\}$
não	sim	não	$\{B\}$
não	sim	sim	$\{B, C\}$
sim	não	não	$\{A\}$
sim	não	sim	$\{A, C\}$
sim	sim	não	$\{A, B\}$
sim	sim	sim	$\{A, B, C\}$

São $2^3 = 8$ subconjuntos. Com 30 elementos, seriam mais de um bilhão.

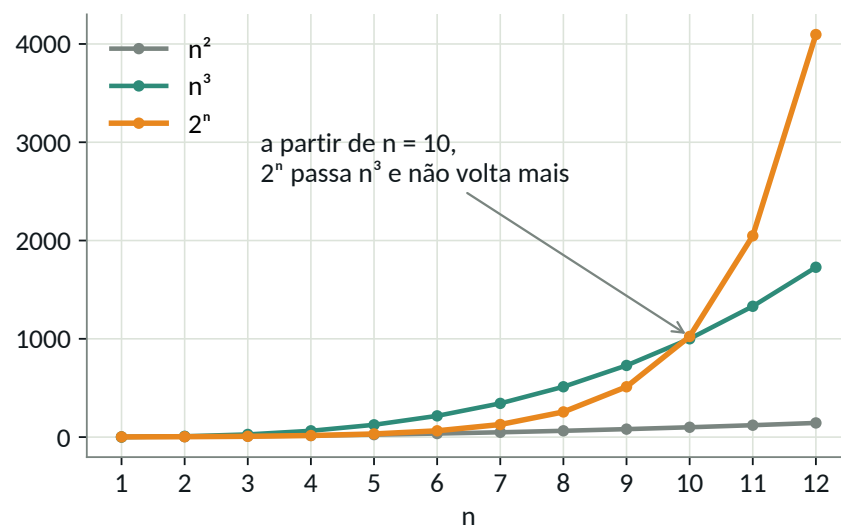


Figura 3.7. No começo, 2^n parece inofensiva: fica abaixo de n^3 até $n = 9$. A partir de $n = 10$, passa à frente e dispara.

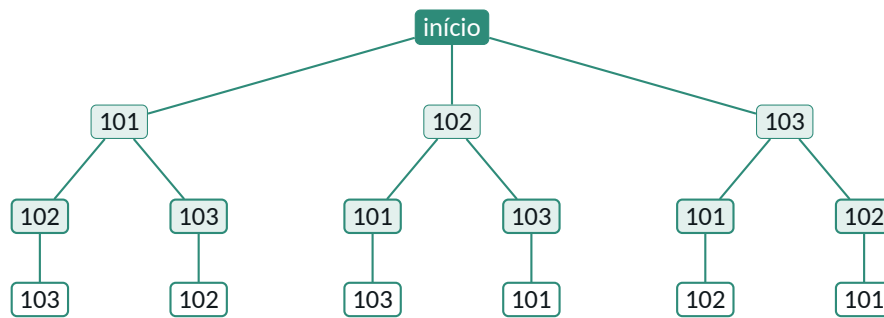


Figura 3.8. As $3! = 6$ ordens de chegada de três trabalhos à fila de impressão. Cada caminho do início até uma folha é uma ordem.

3.6. Fatorial: todas as ordens possíveis

O **fatorial** de n , escrito $n!$ (lê-se “ene fatorial”), é o produto de todos os inteiros de 1 até n :

$$n! = n \times (n-1) \times \cdots \times 2 \times 1, \quad 3! = 3 \cdot 2 \cdot 1 = 6, \quad 5! = 120, \quad 0! = 1 \text{ (por convenção)}.$$

O fatorial conta de quantas **ordens** diferentes podemos arrumar n objetos.

CONEXÃO COM AS AULAS ANTERIORES Na central de impressão da Aula 2, três trabalhos — 101, 102 e 103 — podem chegar à fila em qualquer ordem. Para o primeiro lugar, há 3 opções; escolhido o primeiro, sobram 2 para o segundo; e o último fica com a única que resta: $3 \times 2 \times 1 = 6$ ordens.

n	3	5	10	15	20
2^n	8	32	1.024	32.768	≈ 1 milhão
$n!$	6	120	3.628.800	$\approx 1,3 \times 10^{12}$	$\approx 2,4 \times 10^{18}$

O fatorial cresce ainda mais rápido que a exponencial: a partir de $n = 4$ ($4! = 24 > 2^4 = 16$), passa à frente e nunca mais é alcançado. Um algoritmo que testa todas as ordens possíveis de n itens é inviável já para n em torno de 20.

EXERCÍCIO RÁPIDO 3.4 (a) Liste os subconjuntos de $\{A, B, C\}$ sem olhar a tabela. (b) Calcule $4!$. (c) De quantas formas 4 trabalhos podem chegar à fila? (d) Calcule $\frac{5!}{3!}$ sem calcular os fatoriais inteiros.



3.7. Somatórios e a soma de Gauss

Muitos algoritmos fazem um trabalho que cresce a cada volta de um laço: 1 passo na primeira volta, 2 na segunda, 3 na terceira... Para somar isso tudo, usamos o **somatório**, escrito com a letra grega Σ (sigma maiúsculo):

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n$$

Leia assim: “a soma de i , para i indo de 1 até n ”. O número embaixo diz onde o contador começa; o de cima, onde termina; e a expressão ao lado diz o que somar em cada volta — exatamente como um laço `for`:

```
total = 0
for i in range(1, n + 1):  # i vai de 1 até n
    total = total + i       # soma a expressão "i"
```

Mais exemplos: $\sum_{i=1}^4 i = 1 + 2 + 3 + 4 = 10$ e $\sum_{i=1}^3 2i = 2 + 4 + 6 = 12$.

3.7.1 O truque de Gauss

Conta-se que, criança, Carl Friedrich Gauss recebeu a tarefa de somar os números de 1 a 100 e respondeu em segundos: 5.050. Ele percebeu que, escrevendo a soma de trás para a frente embaixo dela mesma, cada coluna soma sempre 101:

$$\begin{array}{r} 1 + 2 + \dots + 100 \\ + 100 + 99 + \dots + 1 \\ \hline 101 + 101 + \dots + 101 = 100 \times 101 \end{array}$$

Isso é o dobro da soma pedida; então a soma é $\frac{100 \times 101}{2} = 5.050$. Em geral:

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

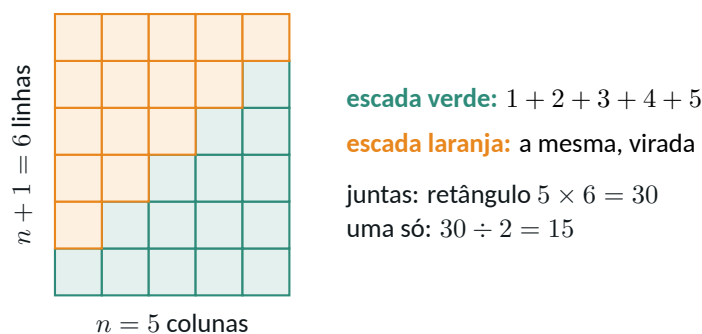


Figura 3.9. A prova visual de Gauss: duas escadas iguais formam um retângulo $n \times (n + 1)$.



Duas variações que aparecem muito:

$$\sum_{i=0}^{n-1} i = 0 + 1 + \dots + (n-1) = \frac{(n-1)n}{2} \quad \text{e} \quad \sum_{i=1}^n c = c \cdot n \quad (\text{somar a constante } c, n \text{ vezes}).$$

CONEXÃO COM AS AULAS ANTERIORES Lembra do exercício 3 da Aula 1 (“contando deslocamentos”)? Inserir n elementos, um a um, sempre no início de uma lista contígua: o primeiro não desloca ninguém; o segundo desloca 1; o terceiro, 2; ...; o último, $n-1$. Total: $0 + 1 + \dots + (n-1) = \frac{n(n-1)}{2}$ deslocamentos. Para $n = 100$, são 4.950.

EXERCÍCIO RÁPIDO 3.5 (a) $1 + 2 + \dots + 100$. (b) Inserindo 10, 20, 30, 40 e 50, sempre no início de uma lista contígua vazia, quantos deslocamentos ocorrem ao todo? (c) E para $n = 100$ inserções? (d) Quanto vale $2 + 4 + 6 + \dots + 2n$? (Dica: coloque o 2 em evidência.)

3.8. Para n muito grande

A análise de algoritmos se interessa pelo que acontece quando n fica **muito grande** — milhões, bilhões. Escrevemos $n \rightarrow \infty$ (lê-se “ene tende ao infinito”) para dizer “ n crescendo sem parar”. Veja o que acontece com duas divisões:

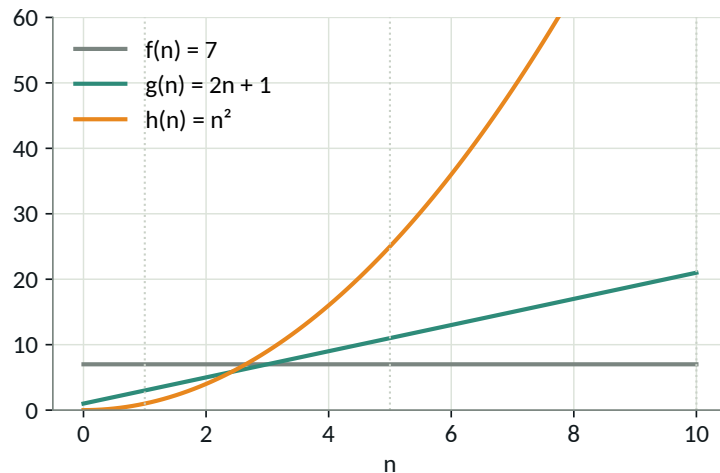
n	10	100	1.000	1.000.000
$\frac{3n+4}{n}$	3,4	3,04	3,004	3,000004
$\frac{n^2}{2^n}$	0,098	$\approx 7,9 \times 10^{-27}$	praticamente 0	praticamente 0

A primeira divisão se aproxima de 3: para n grande, $3n + 4$ se comporta como “3 vezes n ”, e o $+4$ deixa de importar. A segunda se aproxima de 0: 2^n cresce tão mais rápido que n^2 fica insignificante perto dela. Dizemos que a primeira “tende a 3” e a segunda “tende a 0”. Essa é a ideia intuitiva de **limite**, que volta no capítulo 11 — e é exatamente o que o próximo capítulo explora.



Respostas dos exercícios rápidos

3.1. (a) $n = 1$: $f = 7, g = 3, h = 1$. $n = 5$: $f = 7, g = 11, h = 25$. $n = 10$: $f = 7, g = 21, h = 100$.
 (b) Para $n = 1$, a maior é f ; para $n = 5$ e $n = 10$, é h . (c) Quem começa na frente pode terminar atrás: comparar algoritmos com n pequeno engana. O que importa é o comportamento para n grande.



3.2. (a) $2^7 = 128$. (b) 8. (c) n^6 . (d) $\frac{1}{100} = 0,01$. (e) 10, porque $10 \times 10 \times 10 = 1.000$.

3.3. (a) 6, pois $2^6 = 64$. (b) 10. (c) Entre 6 e 7, pois $2^6 = 64 < 100 < 128 = 2^7$ (o valor exato é $\approx 6,64$).
 (d) Cerca de 20 vezes, pois $2^{20} \approx 1.000.000$.

3.4. (a) $\emptyset, \{A\}, \{B\}, \{C\}, \{A, B\}, \{A, C\}, \{B, C\}$ e $\{A, B, C\}$: são 8. (b) $4! = 24$. (c) 24 formas. (d)
 $\frac{5 \cdot 4 \cdot 3!}{3!} = 5 \cdot 4 = 20$.

3.5. (a) $\frac{100 \cdot 101}{2} = 5.050$. (b) $0+1+2+3+4 = 10$, que é $\frac{5 \cdot 4}{2}$. (c) $\frac{100 \cdot 99}{2} = 4.950$. (d) $2+4+\dots+2n =$
 $2(1+2+\dots+n) = 2 \cdot \frac{n(n+1)}{2} = n(n+1)$.



Termo dominante e análise assintótica

NA AULA: slides 17 a 19 | NO REPOSITÓRIO: aula04-complexidade/

No capítulo 2, a soma de um vetor custou $3n + 4$ passos, e os laços aninhados custaram $3n^2 + 4n + 2$. Essas fórmulas são exatas, mas têm detalhes demais. Neste capítulo você vai aprender a ficar só com o que importa.

4.1. Quem manda quando n cresce

Tome $T(n) = 3n^2 + 5n + 2$ e veja quanto cada termo contribui para o total:

n	$3n^2$	$5n$	2	PARTE DE $3n^2$ NO TOTAL
10	300	50	2	85,2%
100	30.000	500	2	98,4%
1.000	3.000.000	5.000	2	99,8%
1.000.000	3×10^{12}	5×10^6	2	99,9998%

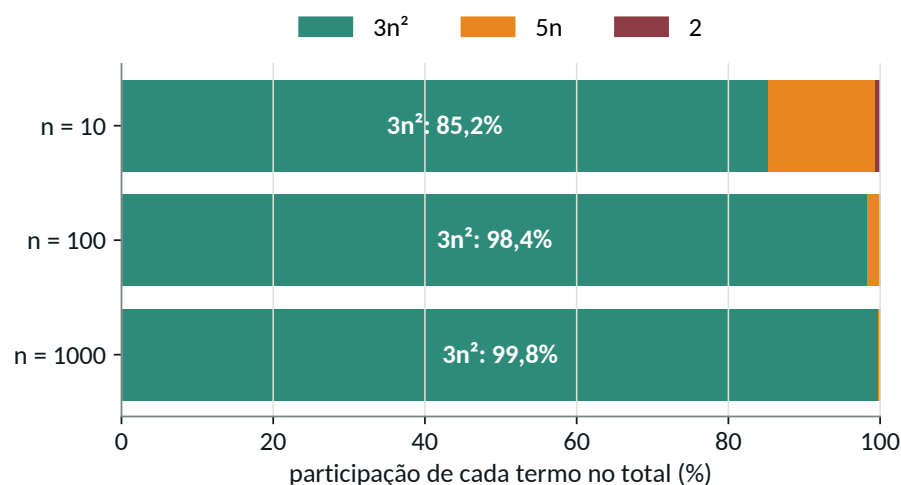


Figura 4.1. Quanto maior n , mais o termo $3n^2$ domina o total.

O termo que cresce mais rápido — aqui, $3n^2$ — acaba respondendo por praticamente todo o custo. Ele é chamado de **termo dominante**. Os outros ficam cada vez mais irrelevantes.



INTUIÇÃO É como comprar um carro e um cafezinho: o preço total é “o preço do carro”. Ninguém deixa de comprar o carro por causa do cafezinho. Em $3n^2 + 5n + 2$, o $3n^2$ é o carro.

4.2. As duas regras de simplificação

1. Fique só com o termo dominante: $3n^2 + 5n + 2 \rightarrow 3n^2$.
2. Despreze a constante que multiplica: $3n^2 \rightarrow n^2$.

Dizemos então que $T(n)$ “cresce como n^2 ”, ou que ele é “quadrático”.

Por que podemos desprezar a constante? Lembre do capítulo 1: rodar o mesmo algoritmo numa máquina mais lenta, ou em Python em vez de C, multiplica o tempo por uma constante (ali, cerca de 35). Constantes mudam de máquina para máquina; a **forma do crescimento** não muda. Como queremos uma medida que valha em qualquer máquina, deixamos a constante de lado.

CUSTO EXATO	TERMO DOMINANTE	CRESCE COMO
$3n + 4$	$3n$	n
$3n^2 + 4n + 2$	$3n^2$	n^2
$2 \log_2 n + 2$	$2 \log_2 n$	$\log n$
$\frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$	$\frac{n^2}{2}$	n^2
$2^n + n^3$	2^n	2^n
500	500	constante

ATENÇÃO Para achar o termo dominante, use a hierarquia do capítulo 3: uma constante cresce menos que $\log n$, que cresce menos que n , que cresce menos que n^2 , que cresce menos que 2^n , que cresce menos que $n!$. Na dúvida, calcule os termos para um n grande, como $n = 1.000$.

4.3. Análise assintótica

Análise assintótica é o nome técnico de “olhar o comportamento quando n fica muito grande”. A palavra vem de *assíntota*, uma linha da qual uma curva se aproxima cada vez mais.

Veja por que isso importa. Imagine dois algoritmos: A , que faz $100n$ passos, e B , que faz n^2 . Para $n = 10$, A faz 1.000 passos e B só 100: B parece melhor! Mas:

n	10	100	1.000	1.000.000
$A(n) = 100n$	1.000	10.000	100.000	10^8
$B(n) = n^2$	100	10.000	1.000.000	10^{12}

Para um milhão de elementos, B faz **dez mil vezes** mais trabalho que A . A constante 100 de A assustava no começo, mas perdeu completamente a importância. Por isso comparamos algoritmos pela forma do crescimento: **para n grande, quem cresce mais devagar sempre vence**, qualquer que seja a constante.

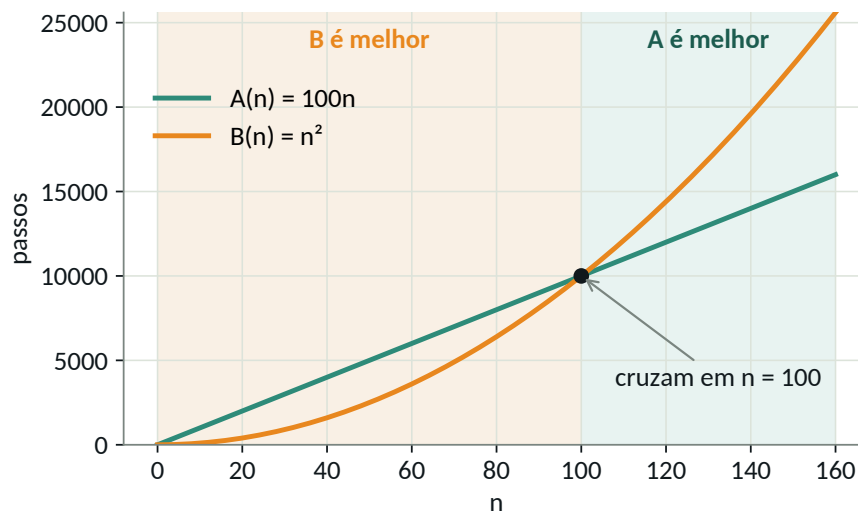


Figura 4.2. $100n$ e n^2 se cruzam em $n = 100$. Depois disso, A ganha, e a vantagem só aumenta.

4.4. Um cuidado: desprezar não é ignorar sempre

A análise assintótica fala de n grande. Para entradas pequenas, as constantes podem, sim, decidir quem é mais rápido. É por isso que bibliotecas reais combinam algoritmos: o `sort()` do Python, por exemplo, usa um algoritmo “de família menor” para listas grandes, mas troca para o Insertion Sort (capítulo 9) em trechos pequenos, onde ele é mais rápido na prática.

EXERCÍCIO RÁPIDO 4.1 Diga como cresce cada função: (a) $4n + 10$; (b) $\frac{n^2}{2} + 3n$; (c) $5 + 2 \log_2 n$; (d) $2^n + n^3$; (e) 1.000. Depois: com $A(n) = 50n$ e $B(n) = n^2$, a partir de qual n o algoritmo A passa a ser melhor?

Respostas dos exercícios rápidos

4.1. (a) como n ; (b) como n^2 ; (c) como $\log n$; (d) como 2^n ; (e) é constante. Os dois se igualam quando $50n = n^2$, isto é, em $n = 50$. Para $n > 50$, A é melhor.



Notação O: o teto

NA AULA: slides 20 a 23 | NO REPOSITÓRIO: aula04-complexidade/

No capítulo 4, dissemos que $3n + 4$ “cresce como n ”. A notação O (lê-se “ó grande”) é a forma precisa de dizer isso — e é a notação que usamos desde a Aula 1.

5.1. A ideia

Escrever $f(n) = O(g(n))$ significa: f **não cresce mais rápido que** g . A função g funciona como um **teto** para f — um teto que pode ser multiplicado por uma constante e que só precisa valer a partir de algum ponto.

CONEXÃO COM AS AULAS ANTERIORES Pense na Pilha contígua da Aula 2: ela tem uma capacidade máxima. O topo pode estar em qualquer posição, mas nunca passa desse limite. A notação O faz o mesmo com o crescimento de uma função: diz até onde ela pode chegar.

5.2. A definição, palavra por palavra

$f(n) = O(g(n))$ se existem constantes $c > 0$ e $n_0 \geq 1$ tais que
 $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.

PEDAÇO	SIGNIFICADO
“existem constantes”	você pode escolher os números; basta encontrar <i>algum</i> par que funcione
$c > 0$	quanto o teto pode ser “esticado” (é o que permite desprezar constantes)
n_0 (“ene zero”)	a partir de onde a regra precisa valer; antes dele, pode falhar
$f(n) \leq c \cdot g(n)$	f fica abaixo do teto (ou encosta nele)
“para todo $n \geq n_0$ ”	não basta valer em alguns pontos: tem que valer daí em diante, sempre

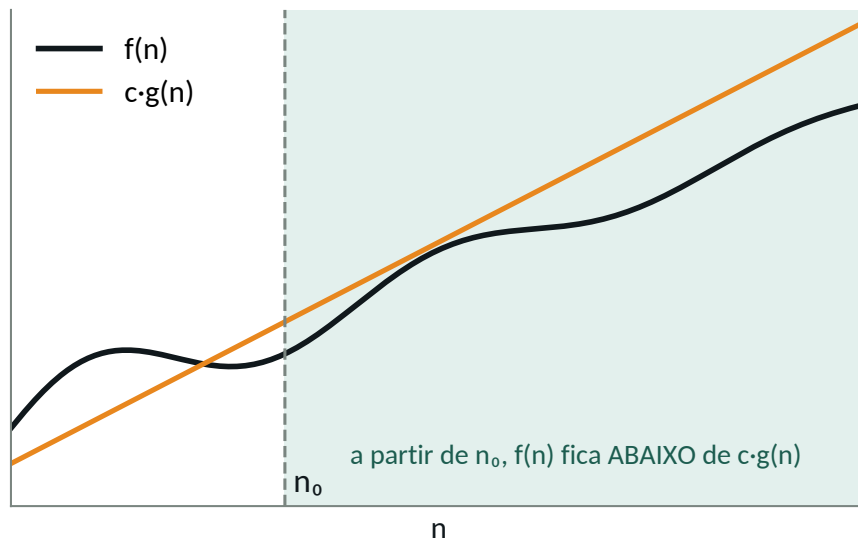


Figura 5.1. Antes de n_0 , f pode até passar do teto. Na região destacada, nunca mais.

5.3. Exemplo guiado: $3n + 4 = O(n)$

Queremos mostrar que existe um par (c, n_0) com $3n + 4 \leq c \cdot n$ para todo $n \geq n_0$.

1. Escolha um c um pouco maior que o coeficiente de n . O coeficiente é 3; vamos tentar $c = 4$.
2. Resolva a desigualdade: $3n + 4 \leq 4n \iff 4 \leq 4n - 3n \iff 4 \leq n$.
3. Leia o resultado: a desigualdade vale para todo $n \geq 4$. Então $n_0 = 4$.

Conferindo com números:

n	1	2	3	4	5	6
$3n + 4$	7	10	13	16	19	22
$4n$	4	8	12	16	20	24
vale $3n + 4 \leq 4n$?	não	não	não	sim	sim	sim

Encontramos $c = 4$ e $n_0 = 4$; logo, $3n + 4 = O(n)$. Outros pares também servem: com $c = 7$, por exemplo, a desigualdade $3n + 4 \leq 7n$ vale desde $n_0 = 1$. A definição só pede que **exista** um par.

5.4. Uma técnica que sempre funciona para polinômios

Para mostrar que $2n^2 + 3n + 1 = O(n^2)$: para $n \geq 1$, temos $n \leq n^2$ e $1 \leq n^2$. Então podemos trocar cada termo pelo maior deles:

$$2n^2 + 3n + 1 \leq 2n^2 + 3n^2 + 1n^2 = 6n^2 \quad \text{para todo } n \geq 1.$$

Logo, $c = 6$ e $n_0 = 1$ funcionam. O truque: **some os coeficientes** e use o resultado como c .

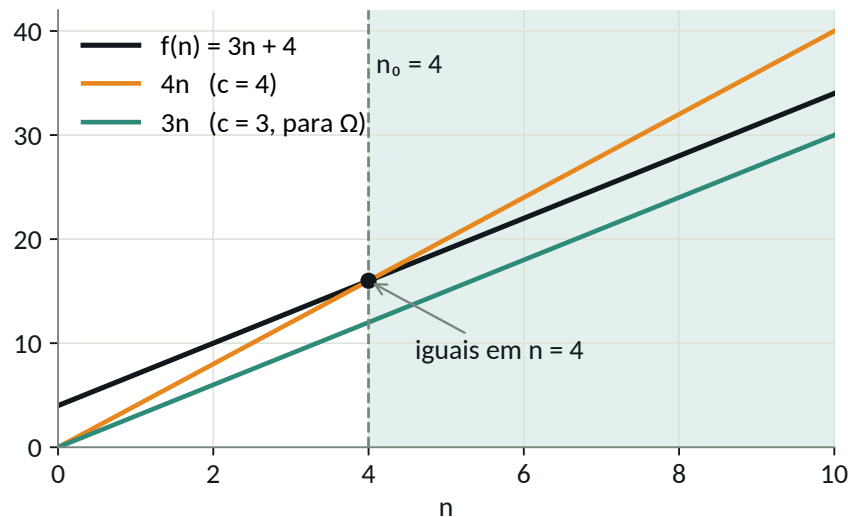


Figura 5.2. $3n + 4$ fica abaixo de $4n$ a partir de $n_0 = 4$. (A reta $3n$ será usada no capítulo 6.)

5.5. Como mostrar que algo NÃO é O

Será que $n^2 = O(n)$? Se fosse, existiriam c e n_0 com $n^2 \leq c \cdot n$ para todo $n \geq n_0$. Dividindo os dois lados por n (que é positivo), isso vira $n \leq c$. Mas n cresce sem parar e c é fixo: basta tomar $n = c + 1$ (e maior que n_0) para a desigualdade falhar. Nenhuma constante segura n^2 debaixo de $c \cdot n$. Portanto, n^2 **não** é $O(n)$.

5.6. O é um teto — e pode ser folgado

É verdade que $3n + 4 = O(n^2)$: um teto quadrático é alto o suficiente para uma função linear. Mas essa informação é pobre, como dizer que “um carro custa menos que um bilhão de reais”. Na prática, sempre buscamos o teto **mais justo** possível: $3n + 4 = O(n)$. Quando quisermos dizer que o teto é exato, usaremos a notação Θ (capítulo 7).

ATENÇÃO Três erros comuns:

- Escrever $O(2n)$ ou $O(n^2 + n)$: as constantes e os termos menores devem ser removidos. Escreva $O(n)$ e $O(n^2)$.
- Achar que $O(n)$ quer dizer “exatamente n passos”. Quer dizer “no máximo proporcional a n , para n grande”.
- Achar que O significa “pior caso”. O é uma forma de comparar funções; o pior caso é uma situação dos dados. Voltaremos a isso nos capítulos 8 e 9.

EXERCÍCIO RÁPIDO 5.1 Mostre que $2n + 10 = O(n)$, encontrando um par c e n_0 .



EXERCÍCIO RÁPIDO 5.2 (a) $7 = O(1)$? Encontre c e n_0 . (b) $n^2 + 100 = O(n^2)$? (c) $n^2 = O(n)$?

Respostas dos exercícios rápidos

5.1. Com $c = 3$: $2n + 10 \leq 3n \iff 10 \leq n$. Então $c = 3$ e $n_0 = 10$. Outra resposta válida: com $c = 12$, $2n + 10 \leq 12n$ vale desde $n_0 = 1$ (pela técnica de somar os coeficientes).

5.2. (a) Sim: $7 \leq 7 \cdot 1$ sempre, com $c = 7$ e $n_0 = 1$. Custo constante é $O(1)$. (b) Sim: $n^2 + 100 \leq n^2 + 100n^2 = 101n^2$ para $n \geq 1$ ($c = 101$, $n_0 = 1$); ou $n^2 + 100 \leq 2n^2$ para $n \geq 10$ ($c = 2$, $n_0 = 10$). (c) Não: veja a seção 5.5.



Notação Ω : o piso

NA AULA: slides 24, 26 e 27 | NO REPOSITÓRIO: aula04-complexidade/

Se O é um teto, Ω (lê-se “ômega grande”) é um **piso**: escrever $f(n) = \Omega(g(n))$ significa que f **cresce pelo menos tão rápido quanto** g .

6.1. A definição

$f(n) = \Omega(g(n))$ se existem constantes $c > 0$ e $n_0 \geq 1$ tais que
 $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$.

É a mesma definição do O , com a desigualdade virada: agora f fica **acima** de $c \cdot g$ a partir de n_0 .

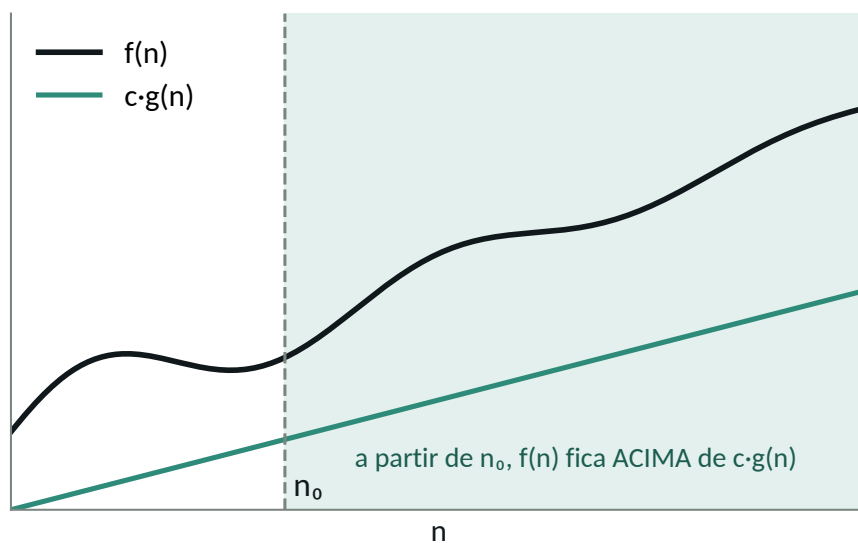


Figura 6.1. A partir de n_0 , f nunca desce abaixo do piso $c \cdot g(n)$.



6.2. Exemplos

$3n + 4 = \Omega(n)$. Com $c = 3$: $3n + 4 \geq 3n$ vale para todo $n \geq 1$ (o $+4$ só ajuda). Então $c = 3$ e $n_0 = 1$. (É a reta $3n$ da figura 5.2.)

$\frac{n^2}{2} - 3n = \Omega(n^2)$. Aqui há um termo *subtraindo*, então precisamos de um pouco mais de cuidado. Vamos tentar $c = \frac{1}{4}$:

$$\frac{n^2}{2} - 3n \geq \frac{n^2}{4} \iff \frac{n^2}{4} \geq 3n \iff n \geq 12.$$

Então $c = \frac{1}{4}$ e $n_0 = 12$ funcionam. A ideia: “gastamos” metade do termo dominante para cobrir o termo negativo, que é menor.

6.3. Ω também descreve problemas

A notação Ω tem um uso muito poderoso: dizer que um **problema** exige, no mínimo, certo trabalho, qualquer que seja o algoritmo.

INTUIÇÃO Achar o maior valor de uma lista **não ordenada** com n elementos exige olhar todos os n . Por quê? Se algum algoritmo deixasse de olhar um único elemento, esse elemento poderia ser justamente o maior, e o algoritmo erraria. Então todo algoritmo correto para esse problema faz pelo menos n leituras: o problema é $\Omega(n)$. Nenhuma ideia genial, em nenhuma linguagem, consegue fazer melhor.

6.4. Como mostrar que algo NÃO é Ω

Será que $n = \Omega(n^2)$? Precisaríamos de $n \geq c \cdot n^2$, ou seja, $1 \geq c \cdot n$, ou ainda $n \leq \frac{1}{c}$. Como n cresce sem parar, isso falha para n grande. Então n **não** é $\Omega(n^2)$: uma função linear não cresce tão rápido quanto uma quadrática.

EXERCÍCIO RÁPIDO 6.1 (a) Mostre que $5n - 20 = \Omega(n)$. (b) $n = \Omega(n^2)$? (c) $n^2 = \Omega(n)$?

Respostas dos exercícios rápidos

6.1. (a) Com $c = 1$: $5n - 20 \geq n \iff 4n \geq 20 \iff n \geq 5$. Então $c = 1$ e $n_0 = 5$. (b) Não (seção 6.4). (c) Sim: $n^2 \geq 1 \cdot n$ para todo $n \geq 1$. Um piso baixo é sempre verdadeiro, mas, como o teto alto do O , é pouco informativo.



Notação Θ : o crescimento exato

NA AULA: slides 25 a 27 | NO REPOSITÓRIO: aula04-complexidade/

Quando uma função tem o mesmo g como teto e como piso, sabemos exatamente como ela cresce. É isso que a notação Θ (lê-se “teta grande”) expressa.

7.1. A definição

$f(n) = \Theta(g(n))$ se existem constantes $c_1 > 0$, $c_2 > 0$ e $n_0 \geq 1$ tais que
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ para todo $n \geq n_0$.

Em outras palavras: $f(n) = \Theta(g(n))$ quando $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$. A função fica “ensanduichada” entre dois múltiplos de g .

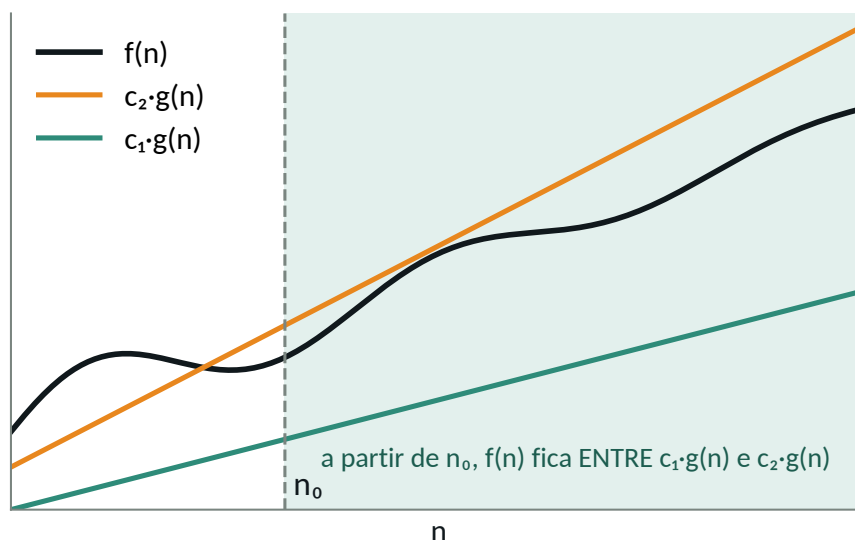


Figura 7.1. A partir de n_0 , f fica presa na faixa entre $c_1 \cdot g(n)$ e $c_2 \cdot g(n)$.



7.2. Exemplos

$3n + 4 = \Theta(n)$. Já temos as duas metades: no capítulo 5, $3n + 4 \leq 4n$ para $n \geq 4$; no capítulo 6, $3n + 4 \geq 3n$ para $n \geq 1$. Juntando, com $c_1 = 3$, $c_2 = 4$ e $n_0 = 4$ (o maior dos dois n_0):

$$3n \leq 3n + 4 \leq 4n \quad \text{para todo } n \geq 4.$$

$n^2 + n = \Theta(n^2)$. Para $n \geq 1$: $n^2 \leq n^2 + n$ (óbvio) e $n^2 + n \leq n^2 + n^2 = 2n^2$. Então $c_1 = 1$, $c_2 = 2$ e $n_0 = 1$.

7.3. Resumo das três notações

NOTAÇÃO	LÊ-SE	IDEIA	DESIGUALDADE	COMO COMPARAR NÚMEROS
$O(g)$	ó grande	teto	$f \leq c \cdot g$	como " $\leq$ "
$\Omega(g)$	ômega grande	piso	$f \geq c \cdot g$	como " $\geq$ "
$\Theta(g)$	teta grande	crescimento exato	$c_1 g \leq f \leq c_2 g$	como " $=$ "

INTUIÇÃO A comparação com números ajuda a lembrar: dizer $f = O(g)$ é como dizer "a velocidade de crescimento de f é $\leq$ a de g "; Ω é " $\geq$ "; e Θ é " $=$ ". Sempre em termos de *forma de crescimento*, ignorando constantes.

7.4. Uma regra prática para polinômios

Se $f(n)$ é um polinômio cujo maior expoente é d e o coeficiente desse termo é positivo, então $f(n) = \Theta(n^d)$. Exemplos: $3n + 4 = \Theta(n)$; $3n^2 + 4n + 2 = \Theta(n^2)$; $\frac{n(n-1)}{2} = \Theta(n^2)$. É a formalização exata da ideia de termo dominante do capítulo 4.

CONEXÃO COM AS AULAS ANTERIORES Agora dá para dizer com precisão alguns custos das aulas anteriores:

- `empilhar()`, `desempilhar()` e `consultar_topo()` da Pilha: $\Theta(1)$.
- `enqueue()` e `dequeue()` da Fila com ponteiro para o fim: $\Theta(1)$.
- `percorrer()` de qualquer Lista com n elementos: $\Theta(n)$.
- Inserir n elementos, um a um, no início de uma lista contígua: $\frac{n(n-1)}{2}$ deslocamentos, ou seja, $\Theta(n^2)$.

EXERCÍCIO RÁPIDO 7.1 (a) Mostre que $3n + 4 = \Theta(n)$, dando c_1 , c_2 e n_0 . (b) Mostre que $4n^2 - 10n + 7 = \Theta(n^2)$. (Dica para o piso: $7 \geq 0$ e "gaste" metade do $4n^2$ com o $-10n$.)



Respostas dos exercícios rápidos

7.1. (a) $c_1 = 3$, $c_2 = 4$, $n_0 = 4$ (seção 7.2). (b) Teto: para $n \geq 1$, $-10n \leq 0$ e $7 \leq 7n^2$, então $4n^2 - 10n + 7 \leq 4n^2 + 7n^2 = 11n^2$. Piso: como $7 \geq 0$, temos $4n^2 - 10n + 7 \geq 4n^2 - 10n$, e $4n^2 - 10n \geq 2n^2 \iff 2n^2 \geq 10n \iff n \geq 5$. Logo, $c_1 = 2$, $c_2 = 11$ e $n_0 = 5$.



Melhor, pior e caso médio

NA AULA: slides 28 | NO REPOSITÓRIO: aula04-complexidade/

No capítulo 2, a busca linear custou 4 passos quando o valor estava na primeira posição e $3n + 3$ quando não estava na lista. O mesmo algoritmo, com o mesmo n , tem custos muito diferentes: tudo depende dos dados. Este capítulo organiza essa ideia.

8.1. Três perguntas diferentes

Para uma entrada de tamanho n , podemos perguntar:

- **Melhor caso:** qual é o *menor* custo possível, escolhendo os dados mais favoráveis?
- **Pior caso:** qual é o *maior* custo possível, com os dados mais desfavoráveis?
- **Caso médio:** qual é o custo *esperado*, em média, para dados típicos?

Cada resposta é uma função de n diferente. Vamos calculá-las para a busca linear, contando só as comparações `dados[i] == valor`.

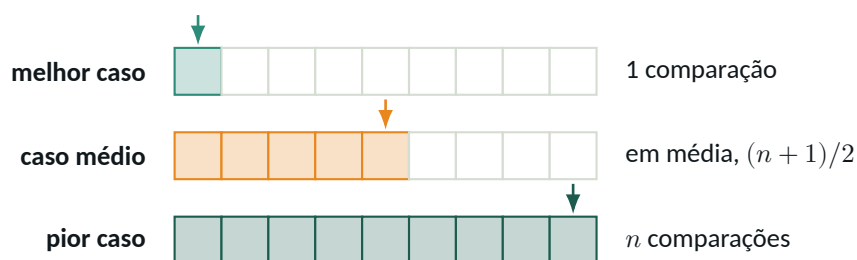


Figura 8.1. Busca linear num vetor de 9 posições: as células coloridas são as comparadas até achar o valor (seta).

8.2. A busca linear, caso a caso

Melhor caso: o valor está na posição 0. Uma comparação e pronto: custo $\Theta(1)$.

Pior caso: o valor está na última posição ou não está na lista. São n comparações: custo $\Theta(n)$.

Caso médio: suponha que o valor está na lista e que qualquer posição é igualmente provável. Se ele está na posição 1, fazemos 1 comparação; na posição 2, fazemos 2; ...; na posição n , fazemos n . A média é a soma dividida por n — e a soma é a de Gauss:

$$\text{média} = \frac{1 + 2 + \dots + n}{n} = \frac{n(n + 1)/2}{n} = \frac{n + 1}{2}.$$



Em média, olhamos cerca de metade da lista. Mas “metade de n ” ainda cresce como n : o caso médio também é $\Theta(n)$.

CASO	COMPARAÇÕES	CRESCIMENTO	QUANDO ACONTECE
Melhor	1	$\Theta(1)$	valor na primeira posição
Médio	$\frac{n+1}{2}$	$\Theta(n)$	valor numa posição qualquer
Pior	n	$\Theta(n)$	valor no fim ou ausente

8.3. Qual caso importa mais?

- O **pior caso** é uma *garantia*: o algoritmo nunca será mais lento que isso. É o mais usado, principalmente em sistemas em que atrasar não é aceitável.
- O **caso médio** descreve o desempenho típico, mas exige supor como os dados costumam ser — e nem sempre sabemos.
- O **melhor caso** raramente serve para escolher um algoritmo (é fácil ter sorte), mas mostra o mínimo que o algoritmo faz e, às vezes, revela situações práticas importantes, como dados quase ordenados (capítulo 9).

8.4. Mais dois exemplos das aulas anteriores

O `conjunto_inserir()` da Aula 3 chama `pertence()` antes de inserir, para não duplicar. Num Conjunto com n elementos:

- melhor caso: o valor já é o primeiro elemento; `pertence()` acha na primeira comparação e a inserção é recusada: $\Theta(1)$;
- pior caso: o valor é novo; `pertence()` compara com todos os n e depois insere no fim: $\Theta(n)$.

E se inserirmos n valores *distintos*, um a um, num Conjunto vazio? A primeira inserção compara com 0 elementos, a segunda com 1, ..., a última com $n - 1$: de novo a soma de Gauss, $\frac{n(n-1)}{2}$ comparações, ou seja, $\Theta(n^2)$. É por isso que o `set()` do Python usa outra estrutura (as tabelas de dispersão, assunto de um curso mais adiante).

A função `maior()` do exercício 2.1 fez $3n + 1$ passos no melhor caso e $4n$ no pior. Os números mudam, mas os dois casos são $\Theta(n)$. Nem sempre o caso muda a *família* do crescimento — mas, às vezes, muda, como você verá no capítulo 9.



8.5. Casos e notações não são a mesma coisa

ATENÇÃO É muito comum confundir “ O = pior caso” e “ Ω = melhor caso”. Não é isso!

- **Melhor, pior e médio** são *situações dos dados*. Cada uma gera uma função de n .
- O , Ω e Θ são *formas de descrever o crescimento de uma função*.

Por isso podemos dizer, por exemplo, “o pior caso da busca linear é $\Theta(n)$ ” e “o melhor caso é $\Theta(1)$ ”. O capítulo 9 mostra essa distinção com o Insertion Sort, em que o melhor e o pior caso pertencem a famílias diferentes.

EXERCÍCIO RÁPIDO 8.1 Considere a função abaixo, que verifica se uma lista tem algum valor repetido. (a) Qual é o melhor caso e quantas comparações ele faz? (b) Qual é o pior caso e quantas comparações ele faz? (c) Escreva os dois com a notação Θ .

```
def tem_repetido(v):  
    for i in range(len(v)):  
        for j in range(i + 1, len(v)):  
            if v[i] == v[j]:  
                return True  
    return False
```

EXERCÍCIO RÁPIDO 8.2 Numa busca linear num vetor de 9 elementos, com o valor presente e todas as posições igualmente prováveis, quantas comparações são feitas em média?

Respostas dos exercícios rápidos

8.1. (a) Melhor caso: $v[0] == v[1]$. A primeira comparação já encontra a repetição: 1 comparação. (b) Pior caso: não há nenhum valor repetido, e todas as duplas são comparadas. Para $i = 0$, são $n - 1$ comparações; para $i = 1, n - 2; \dots$; no total, $(n - 1) + (n - 2) + \dots + 1 + 0 = \frac{n(n - 1)}{2}$. (c) Melhor caso $\Theta(1)$; pior caso $\Theta(n^2)$.

8.2. $\frac{9 + 1}{2} = 5$ comparações.



Estudo de caso: Insertion Sort

NA AULA: slides 29 a 36 | NO REPOSITÓRIO: aula04-complexidade/

Este capítulo junta tudo o que vimos: contagem de operações, somatórios, as notações e os casos. O personagem é um algoritmo de ordenação simples e muito usado na prática, o **Insertion Sort** (ordenação por inserção), cujo custo depende fortemente de como os dados chegam.

9.1. A ideia: arrumar cartas na mão

Pense em como você arruma cartas de baralho recebidas uma a uma: a parte da mão que você já arrumou fica em ordem; quando chega uma carta nova, você a compara com as da direita para a esquerda, abre espaço e a encaixa no lugar certo. O Insertion Sort faz exatamente isso num vetor:

1. A parte da esquerda do vetor (no começo, só o primeiro elemento) já está ordenada.
2. Pegamos o próximo elemento — a **chave**.
3. Deslocamos para a direita todos os elementos ordenados maiores que a chave.
4. Colocamos a chave no espaço que sobrou.

CONEXÃO COM AS AULAS ANTERIORES Os passos 3 e 4 são o `inserir(pos, valor)` da lista contígua da Aula 1: para inserir no meio, é preciso deslocar os elementos seguintes. O Insertion Sort é uma sequência de “inserções no lugar certo”, e o custo dele vem justamente desses deslocamentos.

9.2. O código

```
def insertion_sort(v):
    for i in range(1, len(v)):
        chave = v[i]
        j = i - 1
        while j >= 0 and v[j] > chave:
            v[j + 1] = v[j]      # desloca para a direita
            j -= 1
        v[j + 1] = chave        # encaixa a chave
```



```
void insertion_sort(int v[], int n) {
    for (int i = 1; i < n; i++) {
        int chave = v[i];
        int j = i - 1;
        while (j >= 0 && v[j] > chave) {
            v[j + 1] = v[j];    /* desloca para a direita */
            j--;
        }
        v[j + 1] = chave;      /* encaixa a chave */
    }
}
```

Para analisar, vamos contar duas coisas: as **comparações** (cada avaliação de $v[j] > \text{chave}$) e os **deslocamentos** (cada $v[j + 1] = v[j]$).

9.3. Passo a passo

5	2	4	6	1	3	início: a parte ordenada é só o 5
2	5	4	6	1	3	passada 1, chave 2: 1 comparação, 1 deslocamento
2	4	5	6	1	3	passada 2, chave 4: 2 comparações, 1 deslocamento
2	4	5	6	1	3	passada 3, chave 6: 1 comparação, 0 deslocamentos
1	2	4	5	6	3	passada 4, chave 1: 4 comparações, 4 deslocamentos
1	2	3	4	5	6	passada 5, chave 3: 4 comparações, 3 deslocamentos

Figura 9.1. Ordenando [5, 2, 4, 6, 1, 3]. Em verde, a parte ordenada; em laranja, onde a chave foi encaixada. Total: 12 comparações e 9 deslocamentos.

Repare na passada 4: a chave 1 é menor que todos os ordenados, então é comparada com os quatro, todos se deslocam, e o laço para porque j chega a -1 (sem uma quinta comparação). Na passada 3, a chave 6 já é maior que o último ordenado: uma comparação, nenhum deslocamento.

9.4. Melhor caso: o vetor já está ordenado

1	2	3	4	5		5	4	3	2	1	
1	2	3	4	5	1 comp., 0 desl.	4	5	3	2	1	1 comp., 1 desl.
1	2	3	4	5	1 comp., 0 desl.	3	4	5	2	1	2 comp., 2 desl.
1	2	3	4	5	1 comp., 0 desl.	2	3	4	5	1	3 comp., 3 desl.
1	2	3	4	5	1 comp., 0 desl.	1	2	3	4	5	4 comp., 4 desl.
Melhor caso: [1, 2, 3, 4, 5]						Pior caso: [5, 4, 3, 2, 1]					

Figura 9.2. À esquerda, cada chave já está no lugar: uma comparação por passada. À direita, cada chave precisa ir até o início: as passadas custam 1, 2, 3 e 4.

Se o vetor já está em ordem crescente, cada chave é comparada só com o elemento à sua esquerda, descobre que já está no lugar e nenhum deslocamento acontece. São $n - 1$ passadas com 1 comparação



cada:

$$T_{\text{melhor}}(n) = n - 1 \text{ comparações} \implies T_{\text{melhor}}(n) = \Theta(n).$$

9.5. Pior caso: ordem inversa

Se o vetor está em ordem decrescente, cada chave é menor que todos os já ordenados: na passada i , ela é comparada com os i elementos da esquerda e todos se deslocam. Somando as passadas — a soma de Gauss do capítulo 3:

$$T_{\text{pior}}(n) = 1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} \text{ comparações} \implies T_{\text{pior}}(n) = \Theta(n^2).$$

Para $n = 1.000$, são 499.500 comparações — contra 999 do melhor caso.

9.6. Caso médio: ordem aleatória

Com os dados embaralhados, cada chave, em média, é maior que metade dos elementos já ordenados e menor que a outra metade. Então ela anda, em média, metade do caminho: na passada i , cerca de $i/2$ comparações. Somando, chegamos à metade do pior caso:

$$T_{\text{médio}}(n) \approx \frac{1}{2} \cdot \frac{n(n - 1)}{2} \approx \frac{n^2}{4} \implies T_{\text{médio}}(n) = \Theta(n^2).$$

n	ORDENADO	ALEATÓRIO (MEDIDO)	INVERTIDO	$n^2/4$
10	9	30	45	25
100	99	2.607	4.950	2.500
1.000	999	252.249	499.500	250.000

ATENÇÃO Metade do pior caso continua sendo quadrático! Dividir por 2 é multiplicar por uma constante, e constantes não mudam a família. Em média, o Insertion Sort é tão “quadrático” quanto no pior caso — só com a metade do trabalho.

9.7. Vendo na prática

O programa `python/insertion_sort.py` (e a versão em C) faz o passo a passo e conta as comparações nos três cenários. Esta é a saída real:

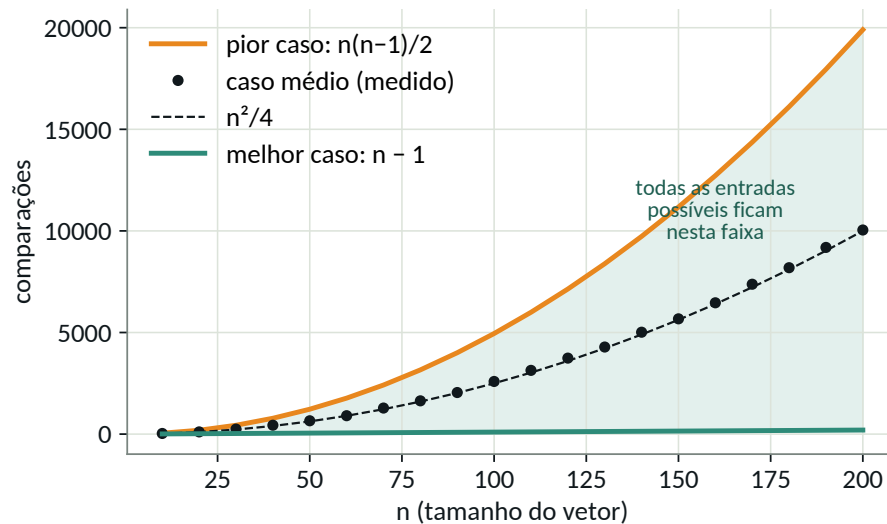


Figura 9.3. Comparações do Insertion Sort nos três cenários. O caso médio (pontos) acompanha $n^2/4$, e o melhor caso quase não sai do chão.

1) INSERTION SORT PASSO A PASSO

vetor inicial: [5, 2, 4, 6, 1, 3]

passada 1 chave = 2 -> [2, 5, 4, 6, 1, 3] comparações: 1 deslocamentos: 1

passada 2 chave = 4 -> [2, 4, 5, 6, 1, 3] comparações: 2 deslocamentos: 1

passada 3 chave = 6 -> [2, 4, 5, 6, 1, 3] comparações: 1 deslocamentos: 0

passada 4 chave = 1 -> [1, 2, 4, 5, 6, 3] comparações: 4 deslocamentos: 4

passada 5 chave = 3 -> [1, 2, 3, 4, 5, 6] comparações: 4 deslocamentos: 3

total: 12 comparações e 9 deslocamentos

2) COMPARAÇÕES POR CENÁRIO

n	ordenado	aleatório(*)	invertido
10	9	30	45
100	99	2607	4950
1000	999	252249	499500

(*) média de vários vetores embaralhados

Formulas: ordenado = $n - 1$ | aleatório $\sim n^2/4$ | invertido = $n(n-1)/2$

Melhor caso: $\Theta(n)$ Caso médio e pior caso: $\Theta(n^2)$

Os números do caso aleatório mudam um pouco entre o Python e o C, porque os dois usam geradores de números aleatórios diferentes. Mas ambos ficam perto de $n^2/4$.

9.8. Onde entram O , Ω e Θ

Este é o ponto que mais confunde — e o Insertion Sort é o exemplo perfeito para esclarecê-lo.

- Cada **caso** é uma função diferente: $T_{\text{melhor}}(n) = n - 1$ e $T_{\text{pior}}(n) = \frac{n(n-1)}{2}$.
- Cada função pode ser descrita com a notação Θ : $T_{\text{melhor}}(n) = \Theta(n)$ e $T_{\text{pior}}(n) = \Theta(n^2)$.
- Para **qualquer** entrada, o custo fica na faixa entre essas duas curvas (a região destacada na figura acima). Então podemos dizer: o custo do Insertion Sort é $O(n^2)$ (nunca passa do pior caso) e $\Omega(n)$ (nunca fica abaixo do melhor caso).



ATENÇÃO Não diga “o Insertion Sort é $\Theta(n^2)$ ” sem dizer de qual caso está falando: no melhor caso ele é $\Theta(n)$! Frases corretas: “o pior caso do Insertion Sort é $\Theta(n^2)$ ”; “o Insertion Sort é $O(n^2)$ ”; “o melhor caso é $\Theta(n)$ ”. E lembre: O não é sinônimo de pior caso, nem Ω de melhor caso — são as notações que usamos para descrever cada caso.

9.9. Dados quase ordenados

O melhor caso não é só uma curiosidade. Muitos dados reais chegam *quase* ordenados: uma lista de chamada com um aluno novo no fim, um histórico em que só o último registro está fora do lugar. Nesses casos, o Insertion Sort faz pouco mais que n comparações.

APROFUNDE (OPCIONAL) Chamamos de **inversão** cada par de elementos fora de ordem (um maior antes de um menor). Cada deslocamento do Insertion Sort desfaz exatamente uma inversão; por isso o número de deslocamentos é igual ao número de inversões do vetor. Um vetor ordenado tem 0 inversões; um invertido tem o máximo, $\frac{n(n-1)}{2}$; um embaralhado tem, em média, $\frac{n(n-1)}{4}$. Por isso o `sort()` do Python (um algoritmo chamado Timsort) usa o Insertion Sort em trechos pequenos: nesses trechos, ele é o mais rápido.

EXERCÍCIO RÁPIDO 9.1 Aplique o Insertion Sort a $[1, 2, 3, 5, 4]$. Quantas comparações e quantos deslocamentos são feitos? O resultado está mais perto do melhor ou do pior caso?

EXERCÍCIO RÁPIDO 9.2 (a) Para $n = 8$, quantas comparações o Insertion Sort faz no melhor caso? E no pior? (b) Quantos deslocamentos são feitos para ordenar $[4, 3, 2, 1]$?

Respostas dos exercícios rápidos

9.1. Passadas 1, 2 e 3 (chaves 2, 3 e 5): 1 comparação cada, nenhum deslocamento. Passada 4 (chave 4): compara com 5 (desloca) e com 3 (para): 2 comparações e 1 deslocamento. Total: 5 comparações e 1 deslocamento — quase o melhor caso ($n - 1 = 4$). O vetor tinha uma única inversão (o par 5, 4).

9.2. (a) Melhor caso: $8 - 1 = 7$; pior caso: $\frac{8 \cdot 7}{2} = 28$. (b) $1 + 2 + 3 = 6$ deslocamentos (é o pior caso para $n = 4$).



Comparando famílias de funções

NA AULA: slides 37 a 40 | NO REPOSITÓRIO: aula04-complexidade/

Até aqui, encontramos custos constantes, logarítmicos, lineares, quadráticos, exponenciais e fatoriais. Este capítulo coloca todos lado a lado e mostra, com números, por que a **família** de uma função importa mais do que qualquer constante ou qualquer computador.

10.1. A hierarquia

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

Leia “<” como “cresce mais devagar que”, para n grande. Cada família tem representantes que você já conhece:

FAMÍLIA	NOME	EXEMPLO
1	constante	<code>consultar_topo()</code> , <code>obter(pos)</code> na lista contígua
$\log n$	logarítmica	adivinhar um número dividindo ao meio; busca binária
$\sqrt{n}$	raiz	busca por saltos em vetor ordenado (capítulo 11)
n	linear	<code>buscar()</code> , <code>percorrer()</code> , achar o maior valor
$n \log n$	“n log n”	os melhores algoritmos de ordenação, como o <code>sort()</code> do Python
n^2	quadrática	Insertion Sort no pior caso, interseção ingênua de conjuntos
n^3	cúbica	três laços aninhados até n
2^n	exponencial	testar todos os subconjuntos de um Conjunto
$n!$	fatorial	testar todas as ordens possíveis de uma fila



10.2. Os números

A tabela abaixo foi gerada pelo programa `python/crescimento_funcoes.py`. Números enormes aparecem em notação científica: $1,3 \times 10^{30}$ quer dizer 1,3 seguido de 30 casas.

FUNÇÃO	$n = 10$	$n = 20$	$n = 50$	$n = 100$	$n = 1.000$
$\log_2 n$	3,3	4,3	5,6	6,6	10,0
$\sqrt{n}$	3,2	4,5	7,1	10,0	31,6
n	10	20	50	100	1.000
$n \log_2 n$	33,2	86,4	282,2	664,4	9.966
n^2	100	400	2.500	10.000	1.000.000
n^3	1.000	8.000	125.000	1.000.000	$1,0 \times 10^9$
2^n	1.024	1.048.576	$1,1 \times 10^{15}$	$1,3 \times 10^{30}$	$1,1 \times 10^{301}$
$n!$	3.628.800	$2,4 \times 10^{18}$	$3,0 \times 10^{64}$	$9,3 \times 10^{157}$	$4,0 \times 10^{2567}$

INTUIÇÃO Um número como $4,0 \times 10^{2567}$ (o valor de $1.000!$) não tem comparação com nada no mundo real: estima-se que o universo inteiro tenha cerca de 10^{80} átomos.

10.3. Lendo um gráfico em escala logarítmica

Se desenharmos essas funções num gráfico comum, $n!$ e 2^n disparam para fora da página e as outras parecem grudadas no chão. A solução é a **escala logarítmica** no eixo vertical: cada marca vale 10 vezes (ou mais) a anterior. Assim, números muito diferentes cabem no mesmo desenho.

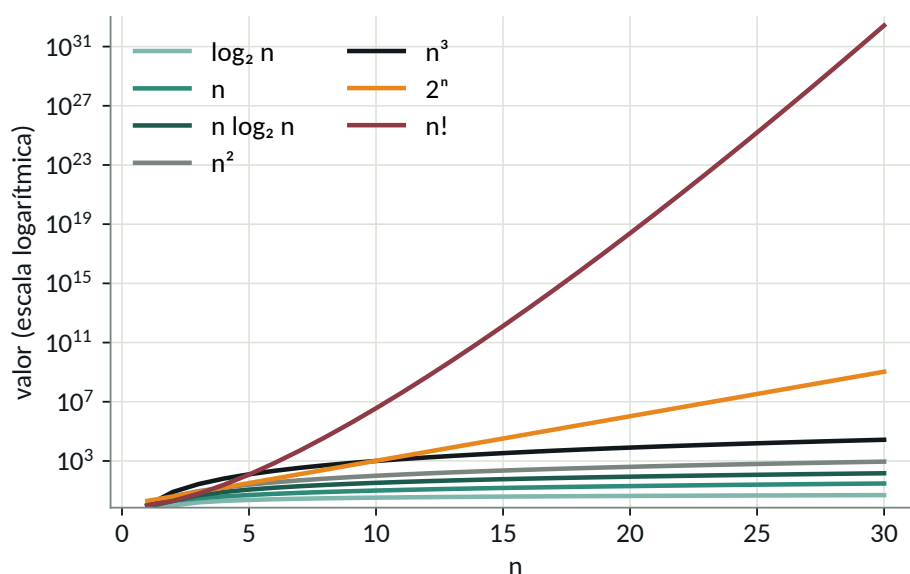


Figura 10.1. As famílias em escala logarítmica. Cada marca do eixo vertical é 10^4 vezes a anterior. $n!$ e 2^n se afastam de todas as outras.



ATENÇÃO Numa escala logarítmica, curvas que parecem próximas podem estar muito distantes. A distância vertical entre duas marcas vizinhas, aqui, é uma multiplicação por 10.000.

10.4. Transformando em tempo

Suponha um computador que execute um passo a cada nanossegundo (um bilionésimo de segundo), ou seja, um bilhão de passos por segundo. Quanto tempo leva cada família?

PASSOS	$n = 10$	$n = 20$	$n = 50$	$n = 100$
n	10 ns	20 ns	50 ns	100 ns
n^2	100 ns	400 ns	2,5 μ s	10,0 μ s
n^3	1,0 μ s	8,0 μ s	125,0 μ s	1,0 ms
2^n	1,0 μ s	1,0 ms	13 dias	$4,0 \times 10^{13}$ anos
$n!$	3,6 ms	77 anos	$9,6 \times 10^{47}$ anos	$3,0 \times 10^{141}$ anos

ns = nanossegundo; μ s = microssegundo (mil ns); ms = milissegundo (um milhão de ns). Para comparar: a idade do universo é de cerca de $1,4 \times 10^{10}$ anos.

Com $n = 100$, um algoritmo cúbico termina em 1 milissegundo; um exponencial levaria 4×10^{13} anos — milhares de vezes a idade do universo.

10.5. E um computador mil vezes mais rápido?

É tentador pensar que hardware melhor resolve tudo. Veja o maior n que cabe em 1 segundo hoje (10^9 passos) e numa máquina 1.000 vezes mais rápida (10^{12} passos):

PASSOS	MAIOR N HOJE	MAIOR N, 1.000× MAIS RÁPIDO	GANHO
n	$1,0 \times 10^9$	$1,0 \times 10^{12}$	1.000 vezes maior
$n \log_2 n$	$4,0 \times 10^7$	$2,9 \times 10^{10}$	726 vezes maior
n^2	31.622	1.000.000	31,6 vezes maior
n^3	1.000	10.000	10,0 vezes maior
2^n	29	39	só +10
$n!$	12	14	só +2

Para um algoritmo linear, a máquina nova resolve problemas 1.000 vezes maiores. Para um quadrático, só 31,6 vezes maiores. Para um exponencial, o ganho é de **10 elementos**; para um fatorial, de **2**. A moral:

Trocar de computador multiplica a velocidade por uma constante.
Trocar de algoritmo pode mudar a família — e isso vale muito mais.



EXERCÍCIO RÁPIDO 10.1 Ordene da que cresce mais devagar para a que cresce mais rápido: n^2 , 2^n , $n \log n$, 100 , $n!$, $\sqrt{n}$, n^3 , $\log n$.

EXERCÍCIO RÁPIDO 10.2 Um algoritmo exponencial (2^n passos) leva 1 segundo para $n = 30$. (a) Quanto tempo leva para $n = 31$? (b) E para $n = 40$? (c) Se o algoritmo fosse quadrático (n^2 passos) e levasse 1 segundo para $n = 30$, quanto levaria para $n = 40$?

Respostas dos exercícios rápidos

10.1. $100 < \log n < \sqrt{n} < n \log n < n^2 < n^3 < 2^n < n!$.

10.2. (a) 2 segundos: cada +1 em n dobra o tempo. (b) São 10 dobras: $2^{10} = 1.024$ segundos, cerca de 17 minutos. (c) O tempo cresce como $(40/30)^2 = 16/9$: cerca de 1,8 segundo.

11

Um toque de Cálculo

NA AULA: slides 41 a 45 | NO REPOSITÓRIO: aula04-complexidade/

Você não precisa ter estudado Cálculo para acompanhar este capítulo. Vamos usar uma única ideia — a **derivada** — e uma única regra para calculá-la. Com elas, dá para medir quanto um custo cresce a cada elemento a mais e até descobrir o melhor valor de um parâmetro de um algoritmo.

11.1. Derivada: a inclinação de uma curva

Nos capítulos anteriores, comparamos funções pelo “quanto crescem”. A derivada torna isso preciso: ela mede a **taxa de crescimento** de uma função em cada ponto — quanto a saída aumenta quando a entrada aumenta um pouquinho.

Graficamente, a derivada de T no ponto n é a **inclinação da reta tangente** (a reta que só “encosta” na curva naquele ponto). Uma inclinação 4 quer dizer: perto daquele ponto, cada unidade a mais em n aumenta T em cerca de 4.

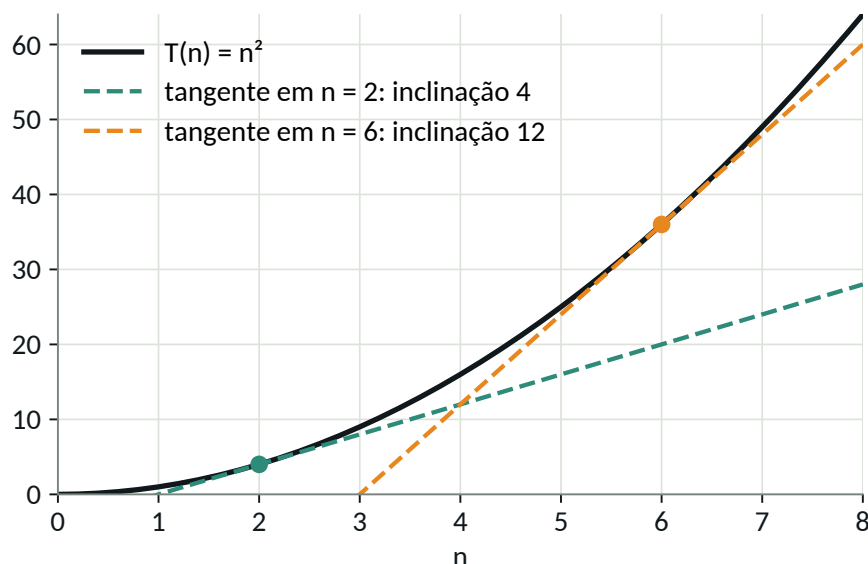


Figura 11.1. Em $T(n) = n^2$, a tangente em $n = 2$ tem inclinação 4; em $n = 6$, inclinação 12. Quanto mais à direita, mais íngreme: o crescimento acelera.

Escrevemos a derivada de $T(n)$ como $T'(n)$ (lê-se “tê linha de ene”). Se $T'(n)$ é constante, a função cresce sempre no mesmo ritmo (é linear); se $T'(n)$ aumenta com n , o crescimento acelera.



11.2. A regra da potência

Para as funções que aparecem em análise de algoritmos, uma regra resolve quase tudo:

$$\frac{d}{dn} (n^p) = p \cdot n^{p-1} \quad (\text{"o expoente desce multiplicando e diminui 1"})$$

FUNÇÃO	DERIVADA	COMO
n	1	$n = n^1 \rightarrow 1 \cdot n^0 = 1$
n^2	$2n$	$2 \cdot n^1$
n^3	$3n^2$	$3 \cdot n^2$
$\sqrt{n} = n^{1/2}$	$\frac{1}{2\sqrt{n}}$	$\frac{1}{2} \cdot n^{-1/2}$
$\frac{1}{n} = n^{-1}$	$-\frac{1}{n^2}$	$-1 \cdot n^{-2}$
uma constante, como 7	0	constantes não variam

Duas regras completam o kit: a derivada de uma **soma** é a soma das derivadas, e uma **constante multiplicando** continua multiplicando. Por exemplo: $\frac{d}{dn} (3n^2 + 5n + 2) = 3 \cdot 2n + 5 \cdot 1 + 0 = 6n + 5$.

11.2.1 A regra da cadeia para u^p

Quando o que está elevado a p não é o próprio n , mas uma expressão u que depende de n , multiplicamos também pela derivada de u :

$$\frac{d}{dn} (u^p) = p \cdot u^{p-1} \cdot u'$$

Exemplos: $\frac{d}{dn} (n+1)^2 = 2(n+1) \cdot 1 = 2n+2$ e $\frac{d}{dn} (3n-2)^2 = 2(3n-2) \cdot 3 = 18n-12$.

11.3. Custo marginal: quanto custa um elemento a mais?

A derivada responde a uma pergunta prática: **se o problema ganhar um elemento, quanto trabalho a mais o algoritmo terá?**

No pior caso do Insertion Sort, $T(n) = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$. Pela regra da potência:

$$T'(n) = \frac{2n}{2} - \frac{1}{2} = n - \frac{1}{2} \approx n.$$

Cada elemento a mais custa cerca de n comparações extras. Conferindo com a conta exata: $T(n+1) - T(n) = \frac{(n+1)n}{2} - \frac{n(n-1)}{2} = n$. Para um vetor de 1.000 elementos, acrescentar mais um custa cerca de 1.000 comparações; para 1.000.000 de elementos, cerca de um milhão.

Compare com a busca linear, $T(n) = n$: $T'(n) = 1$. Cada elemento a mais custa sempre a mesma coisa, uma comparação. **Derivada constante significa crescimento linear; derivada que cresce significa crescimento acelerado.**

11.4. Achando o melhor parâmetro: a busca por saltos

Agora um problema em que a derivada *resolve* uma questão de projeto.

Queremos procurar um valor num vetor **ordenado** com n elementos — por exemplo, uma lista de chamada com 10.000 nomes em ordem alfabética. A busca linear faria até 10.000 comparações. A **busca por saltos** aproveita a ordem e o acesso por índice em $O(1)$ da lista contígua (Aula 1):

1. Salte de m em m posições, olhando só o último elemento de cada bloco, até achar um bloco cujo último elemento seja maior ou igual ao valor procurado.
2. Faça uma busca linear só dentro desse bloco.

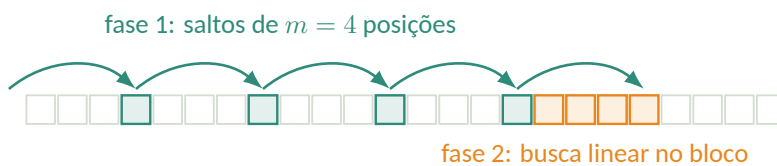


Figura 11.2. Busca por saltos com $m = 4$: os saltos examinam o fim de cada bloco (verde); o valor está no bloco laranja, percorrido um a um.

Qual o melhor tamanho de salto m ? Se m for muito pequeno, damos saltos demais; se for muito grande, o bloco final é enorme. No pior caso, fazemos cerca de n/m saltos mais m comparações dentro do bloco:

$$T(m) = \frac{n}{m} + m.$$

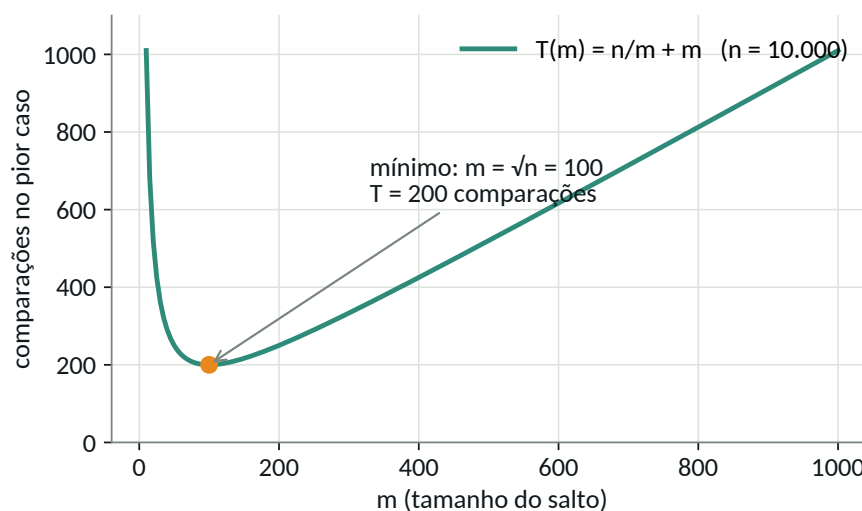


Figura 11.3. $T(m)$ para $n = 10.000$: a curva tem um “vale”. O ponto mais baixo é o melhor salto.

No ponto mais baixo do vale, a curva fica “deitada”: a tangente é horizontal, com inclinação zero. Então basta achar onde $T'(m) = 0$. Escrevendo $\frac{n}{m} = n \cdot m^{-1}$ e usando a regra da potência com $p = -1$:

$$T'(m) = n \cdot (-1) \cdot m^{-2} + 1 = -\frac{n}{m^2} + 1.$$

$$T'(m) = 0 \iff \frac{n}{m^2} = 1 \iff m^2 = n \iff \boxed{m = \sqrt{n}}.$$



Com $n = 10.000$: $m = 100$, e o pior caso fica em $T = 100 + 100 = 200$ comparações — contra 10.000 da busca linear. Em geral, $T(\sqrt{n}) = 2\sqrt{n}$: a busca por saltos é $O(\sqrt{n})$.

O programa `python/busca_saltos.py` testa todos os saltos de 1 a 1.000 e confirma:

```
BUSCA POR SALTOS - comparacoes no pior caso (vetor ordenado)
Modelo: T(m) = n/m + m -> T'(m) = -n/m^2 + 1 = 0 -> m = raiz(n)

n = 10.000 (busca linear, pior caso: 10.000 comparacoes)
  m  pior caso medido  n/m + m
  1      10001      10001
 10      1010       1010
 25       425        425
 50       250        250
100       200        200
200       250        250
400       425        425
1000      1010       1010

Testando todos os m de 1 a 1000:
menor pior caso = 200 comparacoes
obtido com m de 91 a 110 (20 valores empatados)
raiz(10000) = 100 -> 2 * raiz(n) = 200
```

Repare na penúltima linha: vinte valores de m , de 91 a 110, empatam em 200. Perto do mínimo, a curva é quase plana — exatamente o que significa “inclinação zero”.

APROFUNDE (OPCIONAL) Como saber que $m = \sqrt{n}$ é um mínimo, e não um máximo? Pela **segunda derivada** (a derivada da derivada): $T''(m) = \frac{2n}{m^3}$, que é positiva para $m > 0$. Segunda derivada positiva quer dizer que a inclinação está aumentando: a curva tem a forma de um vale, e o ponto de inclinação zero é o fundo.

11.5. Comparando crescimentos com limites

APROFUNDE (OPCIONAL) Às vezes não é óbvio qual de duas funções cresce mais rápido. Por exemplo: $n \log n$ ou $n^{1,5} = n\sqrt{n}$? Uma técnica é olhar o limite da divisão entre elas quando $n \rightarrow \infty$ (capítulo 3): se tende a 0, a de cima cresce mais devagar.

Dividindo as duas por n , sobra comparar $\ln n$ com $\sqrt{n}$. As duas vão para o infinito, e aí entra a **regra de L'Hôpital**: nesse caso, o limite da divisão é igual ao limite da divisão das derivadas.

$$\lim_{n \rightarrow \infty} \frac{\ln n}{n^{1/2}} = \lim_{n \rightarrow \infty} \frac{1/n}{\frac{1}{2}n^{-1/2}} = \lim_{n \rightarrow \infty} \frac{2}{\sqrt{n}} = 0.$$

Conclusão: $n \log n$ cresce mais devagar que $n\sqrt{n}$. (Usamos que a derivada de $\ln n$ é $1/n$.) É por isso que, na hierarquia do capítulo 10, $n \log n$ vem antes de n^2 , e até antes de $n^{1,5}$.

EXERCÍCIO RÁPIDO 11.1 Calcule as derivadas: (a) n^4 ; (b) $5n^2$; (c) $\sqrt{n}$; (d) $\frac{1}{n^2}$; (e) $(n+1)^2$; (f) $(3n-2)^2$.



EXERCÍCIO RÁPIDO 11.2 Na busca por saltos: (a) com $n = 400$, qual é o melhor salto e quantas comparações são feitas no pior caso? (b) E com $n = 1.000.000$? Compare com a busca linear.

EXERCÍCIO RÁPIDO 11.3 Um algoritmo faz $T(n) = n^2 + 3n$ passos. Use a derivada para estimar quantos passos a mais ele faz quando n passa de 100 para 101. Confira calculando $T(101) - T(100)$.

Respostas dos exercícios rápidos

11.1. (a) $4n^3$. (b) $10n$. (c) $\frac{1}{2\sqrt{n}}$. (d) $n^{-2} \rightarrow -2n^{-3} = -\frac{2}{n^3}$. (e) $2(n+1)$. (f) $2(3n-2) \cdot 3 = 18n - 12$.

11.2. (a) $m = \sqrt{400} = 20$; $T = 400/20 + 20 = 40$ comparações (contra 400). (b) $m = 1.000$; $T = 2.000$ comparações (contra 1.000.000).

11.3. $T'(n) = 2n + 3$, então $T'(100) = 203$ passos a mais, aproximadamente. Exato: $T(101) - T(100) = (10.201 + 303) - (10.000 + 300) = 204$. A derivada acerta por pouco.

12

Regras práticas para analisar código

NA AULA: slides 46 | NO REPOSITÓRIO: aula04-complexidade/

Contar cada passo, como no capítulo 2, é ótimo para aprender — mas trabalhoso. Na prática, reconhecemos alguns **padrões** e chegamos direto à notação O (ou Θ).

12.1. Os padrões

PADRÃO	EXEMPLO	CUSTO
comandos simples, sem laço	<code>x = v[0] + 1</code>	$\Theta(1)$
um laço de n voltas com corpo $\Theta(1)$	<code>for i in range(n): soma += v[i]</code>	$\Theta(n)$
laço dentro de laço, n voltas cada	<code>for i ... for j ...</code>	$\Theta(n^2)$
laço “triangular”: o de dentro vai até i	<code>for i in range(n): for j in range(i)</code>	$\Theta(n^2)$ (Gauss)
laço que divide (ou multiplica) por 2	<code>while i > 1: i //= 2</code>	$\Theta(\log n)$
blocos em sequência	bloco A e depois bloco B	soma: fica o maior
<code>if/else</code>	ramo A ou ramo B	o pior dos ramos (para O)
chamada de função dentro de laço	n chamadas de algo $\Theta(m)$	$\Theta(n \cdot m)$

INTUIÇÃO Duas perguntas resolvem quase tudo: **quantas vezes cada laço roda?** e **o que está dentro de quê?** Laços um *depois* do outro somam; laços um *dentro* do outro multiplicam.

12.2. Exemplos resolvidos



Exemplo 1 — laços em sequência e aninhados.

```
for i in range(n):          # bloco A: n * n voltas
    for j in range(n):
        print(i, j)
for k in range(n):          # bloco B: n voltas
    print(k)
```

O bloco A custa n^2 ; o B, n . Em sequência, somam: $n^2 + n$. Fica o termo dominante: $\Theta(n^2)$.

Exemplo 2 — dividindo ao meio.

```
i = n
while i > 1:
    i = i // 2
```

A cada volta, i cai pela metade: o laço roda cerca de $\log_2 n$ vezes. Custo: $\Theta(\log n)$.

Exemplo 3 — o laço triangular.

```
for i in range(n):
    for j in range(i):      # 0, 1, 2, ..., n-1 voltas
        print(i, j)
```

O laço de dentro dá $0 + 1 + 2 + \dots + (n-1) = \frac{n(n-1)}{2}$ voltas no total (Gauss). Custo: $\Theta(n^2)$ — a metade de um quadrado continua quadrática.

Exemplo 4 — um laço logarítmico dentro de um linear.

```
for i in range(n):          # n voltas
    j = 1
    while j < n:              # cerca de log2(n) voltas
        j = j * 2
```

Para cada uma das n voltas de fora, o de dentro roda cerca de $\log_2 n$ vezes. Aninhados, multiplicam: $\Theta(n \log n)$.

Exemplo 5 — a função escondida.

```
def comuns(a, b):           # a tem n elementos, b tem m
    resultado = []
    for x in a:              # n voltas
        if x in b:           # "in" percorre b: O(m)
            resultado.append(x)
    return resultado
```

O `x in b` parece um teste simples, mas é uma busca linear em b (capítulo 2). São n buscas de custo m : $O(n \cdot m)$. É a interseção ingênua da Aula 3.

ATENÇÃO Em Python, desconfie de toda linha que opera sobre uma lista inteira: `in`, `insert(0, x)`, `remove(x)`, `index(x)`, `min`, `max`, `sum`, `sorted`. Cada uma esconde um laço.

**EXERCÍCIO RÁPIDO 12.1** Dê o custo de cada trecho, em notação O .

```
# (a)
for i in range(n):
    for j in range(10):
        print(i, j)

# (b)
i = 1
while i < n:
    i = i * 3

# (c)
for i in range(n):
    lista.insert(0, i)      # lista contígua do Python
```

Respostas dos exercícios rápidos

12.1. (a) O laço de dentro tem sempre 10 voltas (uma constante): $10n$, ou seja, $O(n)$. (b) i é multiplicado por 3 a cada volta: $\log_3 n$ voltas, ou seja, $O(\log n)$ (a base não importa). (c) Cada `insert(0, i)` desloca todos os elementos já inseridos: $0 + 1 + \dots + (n - 1)$ deslocamentos, ou seja, $O(n^2)$.

13

Revisitando Lista, Pilha, Fila e Conjunto

NA AULA: slides 47 | NO REPOSITÓRIO: aula04-complexidade/

Com as ferramentas desta apostila, podemos olhar de novo para as estruturas das Aulas 1 a 3 e dizer, com precisão, quanto custa cada operação. Esta tabela é um ótimo resumo para a prova.

ESTRUTURA	OPERAÇÃO	CUSTO	POR QUÊ
Lista contígua (Aula 1)	<code>obter(pos)</code>	$\Theta(1)$	acesso direto por índice
	inserir/remover no fim	$\Theta(1)$ *	nada a deslocar
	inserir/remover no início	$\Theta(n)$	desloca todos
	<code>buscar(valor)</code>	$O(n)$	melhor caso $\Theta(1)$, pior $\Theta(n)$
Lista encadeada (Aula 1)	inserir/remover no início	$\Theta(1)$	só troca referências
	inserir no fim, com ponteiro <code>fim</code>	$\Theta(1)$	vai direto ao último nó
	<code>obter(pos)</code>	$O(n)$	precisa percorrer os nós
	<code>buscar(valor)</code>	$O(n)$	percorre até achar
Pilha contígua (Aula 2)	empilhar, desempilhar	$\Theta(1)$	só mexe no topo
	<code>consultar_topo()</code>	$\Theta(1)$	acesso direto ao topo
Fila encadeada (Aula 2)	enfileirar (com ponteiro <code>fim</code>)	$\Theta(1)$	insere direto no fim
	desenfileirar	$\Theta(1)$	remove direto do início
Conjunto sobre lista (Aula 3)	<code>pertence(valor)</code>	$O(n)$	percorre os elementos
	<code>inserir(valor)</code> , <code>remover</code>	$O(n)$	chamam <code>pertence()</code>
	união, interseção, diferença	$O(n \cdot m)$	um <code>pertence()</code> por elemento
<code>set()</code> do Python	<code>in</code> , <code>add</code> , <code>remove</code>	$O(1)$ **	tabela de dispersão



* Com espaço disponível (ou, na `list` do Python, em média). ** Em média; usa uma estrutura chamada tabela de dispersão (*hash*), assunto de um curso mais adiante.

13.1. A reflexão da Aula 2, respondida

Na atividade da Aula 2, a pergunta era: *por que manter um ponteiro para o fim permite enfileirar em $O(1)$? O que mudaria se a fila guardasse apenas o início?*

Com o ponteiro `fim`, enfileirar é ir direto ao último nó e ligar o novo: um número fixo de passos, $\Theta(1)$. Sem ele, para achar o último nó seria preciso percorrer a fila inteira a partir do início: $\Theta(n)$ por enfileiramento. E enfileirar n elementos, um a um, custaria $0 + 1 + \dots + (n - 1) = \Theta(n^2)$ — Gauss mais uma vez. No Campo Minado (capítulo 14), a abertura em cascata enfileira até $L \cdot C$ células: com a Fila sem `fim`, ela passaria de $O(L \cdot C)$ para $O((L \cdot C)^2)$.

13.2. Escolhendo a estrutura

- Precisa acessar por posição com frequência? **Lista contígua**.
- Insere e remove muito no início, e raramente acessa por posição? **Lista encadeada**.
- O último a entrar é o primeiro a sair (desfazer, histórico, navegação)? **Pilha**.
- Atendimento por ordem de chegada (impressão, processos, abertura em ondas)? **Fila**.
- Muitas perguntas “este valor está aqui?” e nada de duplicatas? **Conjunto** — e, se forem muitas consultas, um `set()` com custo $O(1)$ em média.

EXERCÍCIO RÁPIDO 13.1 Qual estrutura você usaria, e com que custo por operação, para: (a) o botão “voltar” de um navegador; (b) os pedidos de uma lanchonete, atendidos por ordem de chegada; (c) verificar, milhares de vezes, se um CPF já foi cadastrado?

Respostas dos exercícios rápidos

13.1. (a) Pilha: a última página visitada é a primeira a voltar; empilhar e desempilhar em $\Theta(1)$. (b) Fila: enfileirar e desenfileirar em $\Theta(1)$, com ponteiro para o fim. (c) Um conjunto; com muitas consultas, o `set()` do Python, com `in` em $O(1)$ em média. Com o Conjunto sobre lista da Aula 3, cada consulta custaria $O(n)$.

14

Campo Minado: regras do jogo e técnicas de implementação

NA AULA: slides 48 | NO REPOSITÓRIO: [aula04-complexidade/](#)

A atividade prática desta aula é um Campo Minado completo, que usa as estruturas das Aulas 1 a 3 e as análises da Aula 4. Este capítulo explica as regras do jogo em detalhes e cada técnica usada na solução de exemplo (pasta `solucoes/campo_minado/` do repositório). Os trechos de código são da solução real; o enunciado da atividade diz o que você deve implementar.

14.1. As regras do jogo

14.1.1 O tabuleiro

O jogo acontece num tabuleiro de L linhas e C colunas de **células**. Algumas células — k delas — escondem uma **mina**. No começo, todas estão **fechadas**: o jogador não sabe onde estão as minas. O objetivo é **abrir todas as células que não têm mina**.

NÍVEL	LINHAS × COLUNAS	CÉLULAS	MINAS
Fácil	9 × 9	81	10
Médio	16 × 16	256	40
Difícil	16 × 30	480	99

14.1.2 Os números

Ao abrir uma célula sem mina, ela mostra um **número**: quantas minas existem entre as suas **vizinhas** — as até 8 células que encostam nela, inclusive nas diagonais. Uma célula no meio do tabuleiro tem 8 vizinhas; numa borda, 5; num canto, 3. Os números são as pistas do jogo: um 1 cercado de células abertas, com uma única vizinha fechada, garante que essa vizinha é mina.

14.1.3 A abertura em cascata

Uma célula com 0 não tem nenhuma mina em volta. Então todas as suas vizinhas são seguras, e o jogo as abre automaticamente. Se alguma delas também for 0, as vizinhas dela também se abrem, e assim por diante: uma única jogada pode abrir uma área grande. As células numeradas formam a “borda” da área aberta: elas se abrem, mas não espalham a abertura.



	1	2	3	4	5
1	1	1	2	☠	1
2	1	☠	2	1	1
3	1	1	1	1	1
4	0	0	0	1	☠
5	0	0	0	1	1

Figura 14.1. O tabuleiro 5 × 5 da atividade (Parte A), com minas em (1, 4), (2, 2) e (4, 5) e os números de todas as células seguras.

14.1.4 Bandeiras, primeiro clique, vitória e derrota

- **Bandeira:** o jogador pode marcar uma célula fechada onde acha que há mina. A célula marcada não pode ser aberta (nem pela cascata) até a bandeira ser retirada. O placar mostra “minas menos bandeiras”; ele fica negativo se houver bandeiras demais.
- **Primeiro clique seguro:** as minas só são sorteadas depois do primeiro clique, longe da célula clicada e das suas vizinhas. Assim, o primeiro clique sempre abre uma área.
- **Derrota:** abrir uma célula com mina. O jogo mostra todas as minas e marca as bandeiras colocadas em lugares errados.
- **Vitória:** abrir todas as células seguras. As minas que faltavam ganham bandeira.
- **Desfazer:** só a última bandeira pode ser desfeita. Abrir uma célula não tem volta.

14.1.5 Como jogar nas duas interfaces

Na **interface gráfica** (`jogo_gui.py`): clique esquerdo abre, clique direito (no Mac, Control + clique) marca bandeira, **Ctrl+Z** desfaz a última bandeira e **F2** começa um novo jogo. O menu **Análise** permite escolher Fila ou Pilha para a cascata, animar a abertura, numerar a ordem de abertura e ver o painel com as operações contadas. Na **interface de texto** (`jogo_terminal.py`), os comandos são `r 3 5` (revelar a linha 3, coluna 5), `b 3 5` (bandeira), `d` (desfazer), `h` (histórico), `o` (operações), `a` (alternar Fila e Pilha) e `s` (sair).

14.2. A organização do programa

A decisão de projeto mais importante é **separar a lógica da interface**. Todas as regras ficam na classe `Tabuleiro`, que não imprime nada nem abre janelas. As interfaces só chamam os métodos dela e desenham o resultado.

Vantagens: a mesma lógica serve às duas interfaces; os testes verificam as regras sem abrir janela nenhuma; e dá para trocar a interface sem mexer nas regras (e vice-versa).

14.3. Técnica 1: o tabuleiro numa lista linear

Um tabuleiro parece pedir uma matriz, mas a solução guarda tudo em **listas lineares**, com uma posição por célula — a lista contígua da Aula 1. A célula (linha, coluna) fica na posição

$$\text{índice} = \text{linha} \times C + \text{coluna} \quad (\text{contando a partir de } 0).$$

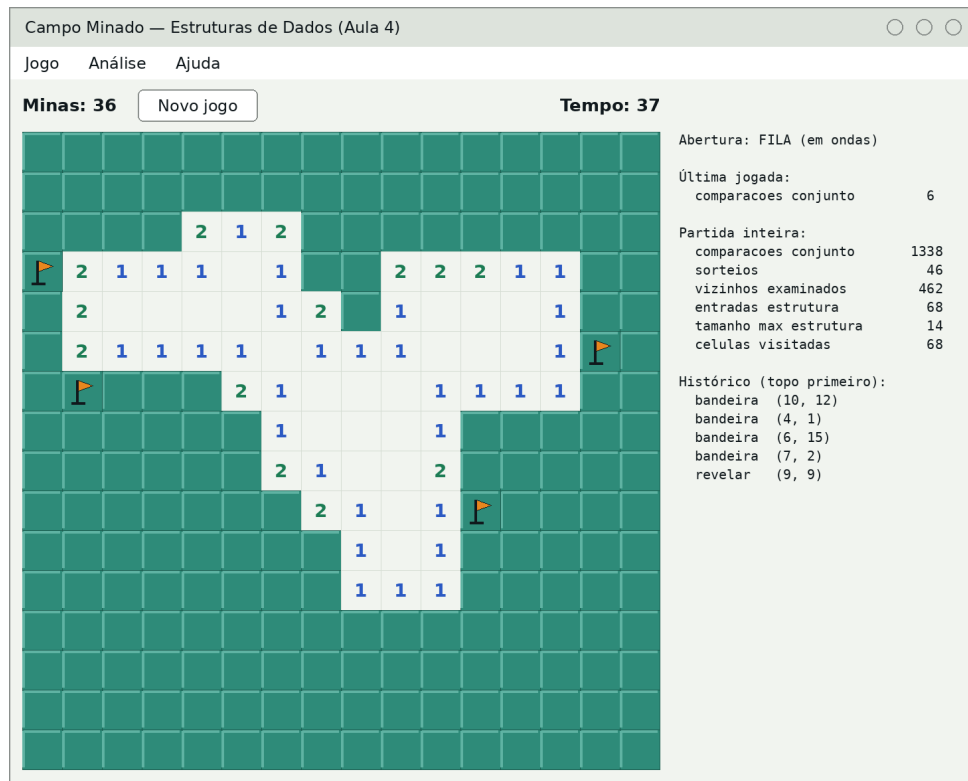


Figura 14.2. A interface gráfica, com o painel de análise (ilustração gerada a partir do código de desenho do jogo).

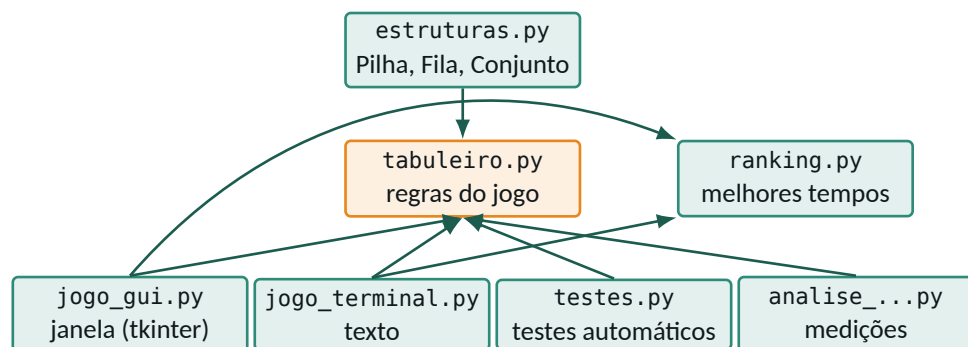


Figura 14.3. Quem usa quem: as setas apontam para o módulo usado. As regras ficam num único lugar.

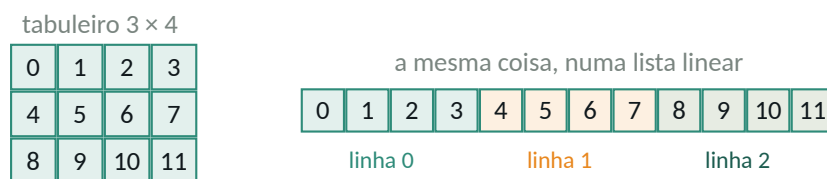


Figura 14.4. Com 4 colunas, a célula (1, 2) fica no índice $1 \times 4 + 2 = 6$.

```

def indice(self, linha, coluna):
    return linha * self.colunas + coluna # O(1)

def coordenadas(self, indice):
    return indice // self.colunas, indice % self.colunas # O(1)
  
```

Ler ou alterar qualquer célula custa $O(1)$. E um único número identifica cada célula, o que simplifica guardar células em Conjuntos, Filas e Pilhas.



14.4. Técnica 2: vizinhança e bordas

As 8 direções ficam numa tupla de deslocamentos, sempre na mesma ordem. Para cada uma, calcula-se a vizinha e descarta-se o que cair fora do tabuleiro:

```

DIRECOES = ((-1, -1), (-1, 0), (-1, 1),      # noroeste, norte, nordeste
            (0, -1), (0, 1),                  # oeste, leste
            (1, -1), (1, 0), (1, 1))          # sudoeste, sul, sudeste

def vizinhos(self, indice):
    linha, coluna = self.coordenadas(indice)
    resultado = []
    for delta_linha, delta_coluna in DIRECOES:
        l, c = linha + delta_linha, coluna + delta_coluna
        if 0 <= l < self.linhas and 0 <= c < self.colunas:  # dentro do tabuleiro?
            resultado.append(self.indice(l, c))
    return resultado

```

São sempre 8 tentativas: custo $\Theta(1)$. A verificação de bordas é essencial. Sem ela, a última célula de uma linha “enxergaria” a primeira célula da linha de baixo, porque na lista linear elas são vizinhas de posição.

14.5. Técnica 3: sortear minas sem repetição

O sorteio usa o **Conjunto** da Aula 3, que já ignora valores repetidos:

```

proibidas = Conjunto(self.contador)          # a célula clicada e suas vizinhas
proibidas.inserir(protegida)
for vizinha in self.vizinhos(protegida):
    proibidas.inserir(vizinha)

while self.minas.tamanho() < self.num_minas:
    candidata = self._gerador.randrange(self.total_celulas)
    if not proibidas.pertence(candidata):
        self.minas.inserir(candidata)          # o Conjunto ignora repetidas

```

Cada `inserir()` chama `pertence()`, que percorre as minas já sorteadas: $0 + 1 + \dots + (k-1) = \frac{k(k-1)}{2}$ comparações, ou seja, $O(k^2)$ — mais as tentativas repetidas, que são poucas quando k é pequeno perto do tabuleiro. No nível Difícil, foram 122 tentativas e 6.515 comparações.

APROFUNDE (OPCIONAL) Uma alternativa sem repetições é o **embaralhamento de Fisher-Yates**: coloque numa lista os índices permitidos e, de trás para a frente, troque cada posição com uma posição sorteada entre as anteriores (inclusive ela). No fim, a lista está embaralhada com todas as ordens igualmente prováveis, e as primeiras k posições são as minas. Custo: $\Theta(L \cdot C)$, sem nenhuma repetição. Vale mais a pena quando há muitas minas, caso em que o sorteio simples repetiria muito.



14.6. Técnica 4: calcular os números percorrendo as minas

A forma óbvia de calcular os números seria perguntar, para cada célula, “quantas das minhas vizinhas são minas?”: $L \cdot C$ células, até 8 consultas ao Conjunto cada uma, e cada consulta custando $O(k)$. Total: $O(L \cdot C \cdot k)$. A solução faz o contrário:

```
for mina in self.minas.valores():           # só as k minas
    for vizinha in self.vizinhos(mina):      # até 8 vizinhas
        self.numeros[vizinha] += 1
```

São no máximo 8 passos por mina: $O(k)$. Medido com o `analise_complexidade.py`:

NÍVEL	$L \times C$	k	PERCORRENDO AS MINAS	PERGUNTANDO A CADA CÉLULA
Fácil	9×9	10	65	5.110
Médio	16×16	40	294	68.664
Difícil	16×30	99	735	317.004

No nível Difícil, a forma ingênua faz cerca de **431 vezes** mais trabalho para chegar ao mesmo resultado. Mudar o ponto de vista do algoritmo mudou a família do custo.

14.7. Técnica 5: a abertura em cascata com Fila ou Pilha

A abertura em cascata é um algoritmo clássico de “preenchimento de região” (*flood fill*). Ele mantém uma estrutura de células **pendentes**: retira uma, e, se ela for 0, coloca na estrutura as vizinhas ainda fechadas.

```
usar_fila = self.estrategia == "fila"
pendentes = Fila() if usar_fila else Pilha(self.total_celulas)
colocar = pendentes.enqueue if usar_fila else pendentes.empilhar
retirar = pendentes.dequeue if usar_fila else pendentes.desempilhar

ordem = []
self._marcar_revelada(inicio)           # marca AO COLOCAR na estrutura
colocar(inicio)
while not pendentes.esta_vazia():
    atual = retirar()
    ordem.append(atual)
    if self.numeros[atual] != 0:
        continue                       # numerada: abre, mas não espalha
    for vizinha in self.vizinhos(atual):
        if not self.reveladas[vizinha] and not self.bandeiras.pertence(vizinha):
            self._marcar_revelada(vizinha)
            colocar(vizinha)
return ordem
```

Três detalhes fazem toda a diferença:

- **Funções como valores:** `colocar` e `retirar` guardam as operações da estrutura escolhida, e o resto do algoritmo é o mesmo para Fila e Pilha.

- **Marcar ao colocar:** a célula é marcada como aberta no momento em que entra na estrutura. Assim, cada célula entra no máximo uma vez, e a abertura custa $O(L \cdot C)$. Se marcássemos só ao retirar, uma célula poderia entrar uma vez para cada vizinha que a visse antes de ela sair:

N	Células (N^2)	ENTRADAS, MARCANDO AO COLOCAR	ENTRADAS, MARCANDO AO RETIRAR
10	100	100	343
20	400	400	1.483
40	1.600	1.600	6.163

Tabuleiro $N \times N$ sem minas, aberto a partir do canto: o pior caso da cascata.

- **Fila ou Pilha:** as duas abrem exatamente as mesmas células, com o mesmo custo de tempo. Muda a **ordem**: a Fila abre em ondas a partir do clique (busca em largura); a Pilha segue um caminho em profundidade antes de voltar (busca em profundidade).

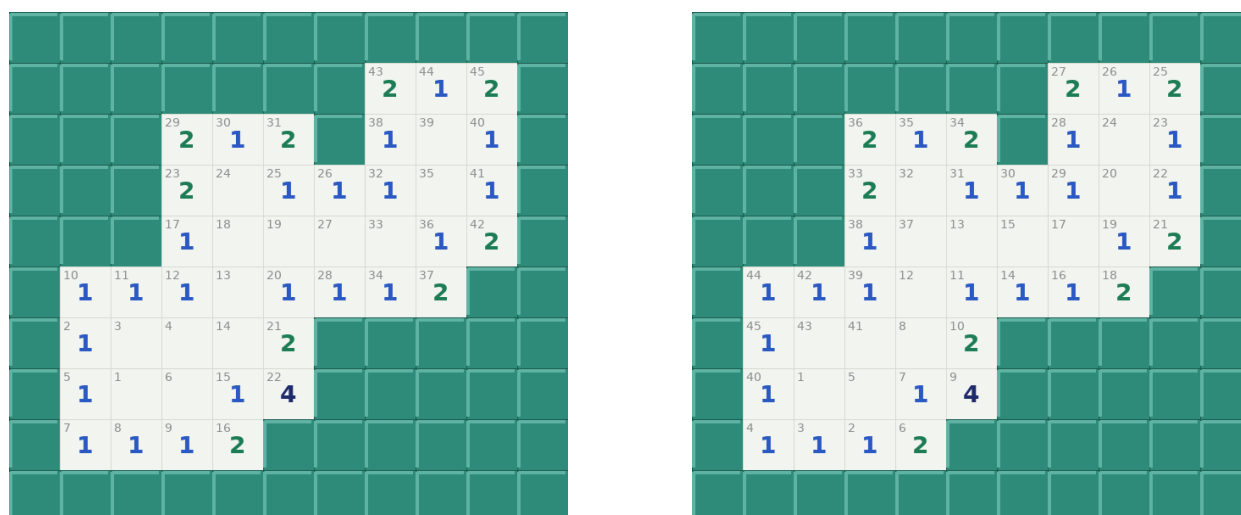


Figura 14.5. Mesmo tabuleiro, mesmo clique: com Fila (esquerda) e com Pilha (direita). Os números pequenos indicam a ordem de abertura.

A diferença aparece na **memória**: a Fila guarda só a “frente da onda”; a Pilha pode acumular muito mais células ao mesmo tempo. Por isso a Pilha contígua (de capacidade fixa, como na Aula 2) é criada com capacidade $L \cdot C$.

N	CÉLULAS	MAIOR TAMANHO DA FILA	MAIOR TAMANHO DA PILHA
10	100	19	44
20	400	39	142
40	1.600	79	487
80	6.400	159	1.777



14.8. Técnica 6: vitória em $O(1)$

Para saber se o jogador venceu, poderíamos varrer a lista `reveladas` a cada jogada: $O(L \cdot C)$ por jogada. A solução mantém um contador, `celulas_seguras_restantes`, que começa em $L \cdot C - k$ e diminui 1 a cada célula aberta. A vitória é simplesmente `celulas_seguras_restantes == 0`: $O(1)$. É a mesma ideia do `tamanho` guardado na Lista da Aula 1: **manter uma informação atualizada custa pouco e evita recalcular tudo**.

14.9. Técnica 7: histórico com Pilha e o desfazer

Cada jogada é empilhada como uma tupla ("`revelar`", `linha`, `coluna`) ou ("`bandeira`", `linha`, `coluna`). Desfazer consulta o **topo**: se for uma bandeira, desempilha e desfaz; se for um "`revelar`", não faz nada, porque abrir uma célula não tem volta.

```
acao, linha, coluna = self.historico.consultar_topo()
if acao != "bandeira":
    return None                                # revelar não tem volta
self.historico.desempilhar()
```

É o comportamento LIFO em ação: só o último elemento é acessível. As "últimas jogadas" mostradas no painel são os valores da Pilha do topo para a base, sem remover nada.

14.10. Técnica 8: ranking com inserção ordenada

O ranking guarda os 10 melhores tempos de cada nível numa lista sempre ordenada. Para inserir um recorde, faz um passo do Insertion Sort: coloca no fim e desloca para a direita os tempos maiores. Custo $O(n)$ no pior caso e $O(1)$ no melhor. Ao carregar o arquivo, ordena com o Insertion Sort completo — e, como o próprio programa salva o arquivo já ordenado, cai no **melhor caso**, $\Theta(n)$, exatamente o cenário do capítulo 9.

14.11. Técnica 9: medir contando operações

A classe `ContadorOperacoes` registra, por categoria, o trabalho de cada jogada: "entradas na estrutura", "células visitadas", "vizinhos examinados", "comparações no Conjunto". O próprio `pertence()` do Conjunto conta cada comparação que faz — por isso ele usa um laço explícito em vez de `valor in lista`, que esconderia o trabalho. É a contagem de operações do capítulo 2, aplicada a um programa inteiro.



14.12. Técnica 10: interfaces, eventos e testes

- **Programação orientada a eventos:** a interface gráfica não tem um laço que lê comandos. Ela registra funções que o tkinter chama quando algo acontece (`canvas.bind("<Button-1>", ...)` para o clique) e usa `raiz.after(ms, funcao)` para agendar ações no futuro: o relógio e cada quadro da animação da abertura.
- **Desenho com etiquetas:** cada célula desenhada no Canvas recebe a etiqueta "`c<indice>`", para ser apagada e redesenhada sozinha, sem redesenhar o tabuleiro todo.
- **Testes com semente fixa:** com `random.Random(mente)`, o sorteio é sempre o mesmo, e os testes podem conferir resultados exatos. O `testes.py` usa o tabuleiro da Parte A e confere, por exemplo, a ordem exata de abertura com Fila e com Pilha.

14.13. Resumo dos custos

OPERAÇÃO	CUSTO	TÉCNICA
<code>indice()</code> , <code>coordenadas()</code> , <code>vizinhos()</code>	$\Theta(1)$	lista linear, 8 direções
sortear as minas	$O(k^2)$	Conjunto sobre lista
calcular os números	$O(k)$	percorrer as minas
abertura em cascata	$O(L \cdot C \cdot f)$	Fila ou Pilha, marcar ao colocar
verificar vitória	$\Theta(1)$	contador de células seguras
bandeira, desfazer	$O(f)$	Conjunto e Pilha
registrar recorde	$O(n)$	inserção ordenada

f = número de bandeiras. Na abertura, cada vizinha examinada consulta o Conjunto de bandeiras ($O(f)$); com uma lista de booleanos, essa consulta seria $O(1)$ e a abertura, $O(L \cdot C)$.

EXERCÍCIO RÁPIDO 14.1 No tabuleiro 5×5 da Parte A (5 colunas, contando a partir de 0 no código): (a) qual é o índice da célula (3, 4)? (b) quais são os índices das vizinhas da célula de índice 0? (c) quantas vizinhas tem a célula de índice 12?

Respostas dos exercícios rápidos

14.1. (a) $3 \times 5 + 4 = 19$ — é a mina da linha 4, coluna 5 (contando a partir de 1). (b) A célula 0 é o canto superior esquerdo: vizinhas 1, 5 e 6. (c) A célula 12 é (2, 2), no meio do tabuleiro: 8 vizinhas.

15

Lista de exercícios resolvidos

NA AULA: slides todos | NO REPOSITÓRIO: aula04-complexidade/

Resolva cada exercício antes de olhar a resolução. As estrelas indicam a dificuldade: ★ direto, ★★ médio, ★★★ desafiador. O bloco D traz questões objetivas no estilo da prova automática do Grau 1 no LEX.

15.1. Bloco A — contagem e matemática

Exercício A1 ★

Conte os passos do trecho abaixo e escreva $T(n)$. Quanto vale $T(100)$?

```
int s = 0;
for (int i = 0; i < n; i++) {
    s = s + i * i;
}
```

RESOLUÇÃO $s = 0$; $i = 0$; testes: $n + 1$; corpo: n ; incrementos: n . Total: $T(n) = 3n + 3$.
Para $n = 100$: $T(100) = 303$.

Exercício A2 ★

Quantas vezes dá o laço `for i in range(0, n, 2)` (de 2 em 2)? Qual é o seu custo?

RESOLUÇÃO Ele passa por 0, 2, 4, ... : cerca de $n/2$ voltas. Metade de n continua crescendo como n (a constante $1/2$ é desprezada): $\Theta(n)$.

Exercício A3 ★

Calcule: (a) $\log_2 512$; (b) $\log_2 2^{20}$; (c) $\log_2 1$; (d) entre quais inteiros está $\log_2 3.000$?

RESOLUÇÃO (a) 9, pois $2^9 = 512$. (b) 20 (o expoente “desce”). (c) 0, pois $2^0 = 1$. (d) Entre 11 e 12, pois $2^{11} = 2.048 < 3.000 < 4.096 = 2^{12}$.



Exercício A4 ★★

Calcule (a) $\sum_{i=1}^{50} i$ e (b) $\sum_{i=1}^n (2i + 1)$.

RESOLUÇÃO (a) $\frac{50 \cdot 51}{2} = 1.275$. (b) Separando: $2 \sum i + \sum 1 = 2 \cdot \frac{n(n+1)}{2} + n = n^2 + n + n = n^2 + 2n$. Confira com $n = 2$: $3 + 5 = 8 = 4 + 4$.

Exercício A5 ★★

Uma lista contígua tem n elementos. Removemos sempre o **primeiro** elemento, até a lista ficar vazia. Quantos deslocamentos acontecem ao todo? Qual é o custo?

RESOLUÇÃO Remover o primeiro de uma lista com t elementos desloca os $t-1$ restantes. Somando para $t = n, n-1, \dots, 1$: $(n-1) + (n-2) + \dots + 0 = \frac{n(n-1)}{2}$ deslocamentos: $\Theta(n^2)$. Com uma Fila encadeada, cada remoção seria $\Theta(1)$ e o total, $\Theta(n)$.

Exercício A6 ★★

A função `tem_repetido()` do capítulo 8 recebe uma lista de 10 elementos sem nenhum valor repetido. Quantas comparações ela faz?

RESOLUÇÃO É o pior caso: todas as duplas são comparadas. $9 + 8 + \dots + 1 + 0 = \frac{10 \cdot 9}{2} = 45$.

15.2. Bloco B — notações

Exercício B1 ★

Mostre que $5n + 3 = O(n)$.

RESOLUÇÃO Com $c = 6$: $5n + 3 \leq 6n \iff n \geq 3$. Então $c = 6, n_0 = 3$. (Pela técnica de somar coeficientes: $5n + 3 \leq 8n$ para $n \geq 1$, com $c = 8, n_0 = 1$.)

Exercício B2 ★★

Mostre que $n^3 + 2n^2 = \Theta(n^3)$.

RESOLUÇÃO Piso: $n^3 + 2n^2 \geq n^3$ sempre. Teto: para $n \geq 1$, $2n^2 \leq 2n^3$, então $n^3 + 2n^2 \leq 3n^3$. Logo, $c_1 = 1, c_2 = 3$ e $n_0 = 1$.



Exercício B3 ★★

Verdadeiro ou falso? (a) $n = O(n^2)$; (b) $n^2 = O(n)$; (c) $2^{n+1} = O(2^n)$; (d) $3^n = O(2^n)$; (e) $\log_2 n = O(\log_{10} n)$; (f) $n \log n = O(n^2)$.

RESOLUÇÃO (a) V (teto folgado, mas verdadeiro). (b) F (seção 5.5). (c) V: $2^{n+1} = 2 \cdot 2^n$, basta $c = 2$. (d) F: $3^n/2^n = 1,5^n$ cresce sem limite; nenhuma constante segura. (e) V: as bases só diferem por uma constante ($\approx 3,32$). (f) V: $\log n \leq n$, então $n \log n \leq n \cdot n$.

Exercício B4 ★★★

Mostre que $n^2 - 10n = \Omega(n^2)$.

RESOLUÇÃO Tentando $c = \frac{1}{2}$: $n^2 - 10n \geq \frac{n^2}{2} \iff \frac{n^2}{2} \geq 10n \iff n \geq 20$. Então $c = \frac{1}{2}$ e $n_0 = 20$.

Exercício B5 ★

Dê o Θ de cada função: (a) $7n^2 + n \log n$; (b) $n + \sqrt{n}$; (c) $2^n + n^{100}$; (d) $(n+1)(n-1)$; (e) $1.000 \log n + 5$.

RESOLUÇÃO (a) $\Theta(n^2)$. (b) $\Theta(n)$. (c) $\Theta(2^n)$ — a exponencial vence qualquer potência, mesmo n^{100} , para n grande o bastante. (d) $n^2 - 1$: $\Theta(n^2)$. (e) $\Theta(\log n)$.

Exercício B6 ★

Por que não se escreve $O(2n)$ nem $O(n^2 + n)$?

RESOLUÇÃO Porque O descreve a forma do crescimento: constantes multiplicativas e termos menores não mudam essa forma. $O(2n)$ e $O(n)$ são o mesmo conjunto de funções; por convenção, usa-se a forma mais simples: $O(n)$ e $O(n^2)$.

15.3. Bloco C — casos e algoritmos

Exercício C1 ★

Aplique o Insertion Sort a $[3, 1, 2]$, contando comparações e deslocamentos.

RESOLUÇÃO Passada 1 (chave 1): compara com 3 (desloca) e j chega a -1 : 1 comparação, 1 deslocamento: $[1, 3, 2]$. Passada 2 (chave 2): compara com 3 (desloca) e com 1 (para): 2 comparações, 1 deslocamento: $[1, 2, 3]$. Total: 3 comparações e 2 deslocamentos (e 2 inversões no vetor original: os pares $(3,1)$ e $(3,2)$).

**Exercício C2 ★**

Numa busca linear em 99 elementos, com o valor presente e qualquer posição igualmente provável, quantas comparações são feitas em média?

RESOLUÇÃO $\frac{n+1}{2} = \frac{100}{2} = 50.$

Exercício C3 ★★

Num vetor **ordenado**, uma busca linear pode parar assim que encontrar um elemento maior que o valor procurado (ele não pode estar mais à frente). Qual é o melhor e o pior caso?

RESOLUÇÃO Melhor caso: o primeiro elemento já é maior ou igual ao valor: 1 comparação, $\Theta(1)$. Pior caso: o valor é maior que todos (ou é o último): n comparações, $\Theta(n)$. A parada antecipada melhora o caso médio quando o valor não está, mas não muda a família do pior caso.

Exercício C4 ★★

Na busca por saltos com $n = 2.500$, qual é o melhor salto e quantas comparações são feitas no pior caso?

RESOLUÇÃO $m = \sqrt{2.500} = 50$; $T = 2.500/50 + 50 = 100$ comparações (contra 2.500 da busca linear).

Exercício C5 ★★★

O **Bubble Sort com parada antecipada** percorre o vetor trocando vizinhos fora de ordem e para quando uma passada inteira não faz nenhuma troca. Qual é o melhor e o pior caso, em comparações?

```
def bubble_sort(v):
    n = len(v)
    for fim in range(n - 1, 0, -1):
        trocou = False
        for i in range(fim):
            if v[i] > v[i + 1]:
                v[i], v[i + 1] = v[i + 1], v[i]
                trocou = True
        if not trocou:
            break
            # nenhuma troca: já está ordenado
```

RESOLUÇÃO Melhor caso (vetor ordenado): a primeira passada faz $n - 1$ comparações, nenhuma troca, e o algoritmo para: $\Theta(n)$. Pior caso (ordem inversa): todas as passadas acontecem, com $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2}$ comparações: $\Theta(n^2)$. Mesmo perfil do Insertion Sort — embora, na prática, o Bubble Sort faça mais trocas.



Exercício C6 ★★

Uma Fila encadeada guarda só o ponteiro para o início. Qual é o custo de enfileirar n elementos, um a um, começando da fila vazia?

RESOLUÇÃO Para enfileirar, é preciso percorrer até o último nó: com t elementos, t passos. Total: $0 + 1 + \dots + (n - 1) = \frac{n(n - 1)}{2}$: $\Theta(n^2)$. Com o ponteiro **fim**, seria $\Theta(n)$ no total (capítulo 13).

15.4. Bloco D — questões objetivas (estilo da prova)

Exercício D1 ★

Qual é o custo de ler $v[i]$ numa lista contígua com n elementos? (a) $O(n)$ (b) $O(1)$ (c) $O(\log n)$ (d) $O(n^2)$

RESOLUÇÃO (b). A posição é calculada diretamente a partir do índice (Aula 1), sem percorrer nada.

Exercício D2 ★

Se $T(n) = 5n^3 + 100n^2 + 7$, então: (a) $T(n) = \Theta(n)$ (b) $T(n) = \Theta(n^2)$ (c) $T(n) = \Theta(n^3)$ (d) $T(n) = \Theta(n^5)$

RESOLUÇÃO (c). O termo dominante é $5n^3$; desprezando a constante, $\Theta(n^3)$. O 100 do segundo termo assusta, mas não muda a família.

Exercício D3 ★★

Sobre o Insertion Sort, é correto afirmar:

- (a) seu custo é $\Theta(n^2)$ para qualquer entrada;
- (b) seu melhor caso ocorre com o vetor já ordenado e é $\Theta(n)$;
- (c) seu pior caso é $\Theta(n \log n)$;
- (d) seu custo não depende da ordem inicial dos dados.

RESOLUÇÃO (b). (a) é falsa: no melhor caso ele é $\Theta(n)$. (c) é falsa: o pior caso é $\Theta(n^2)$. (d) é falsa: a ordem inicial é justamente o que separa o melhor do pior caso.

Exercício D4 ★★

Um algoritmo que faz 2^n passos leva 1 minuto para $n = 40$. Quanto leva para $n = 42$? (a) 1 minuto (b) 2 minutos (c) 4 minutos (d) 42 minutos

RESOLUÇÃO (c). Cada $+1$ em n dobra o tempo: duas dobros, $1 \times 2 \times 2 = 4$ minutos.

**Exercício D5** ★

Quantas vezes dá o laço `while i > 1: i = i // 2`, começando com `i = 1024`? (a) 10 (b) 512
(c) 1.024 (d) 32

RESOLUÇÃO (a). $1024 \rightarrow 512 \rightarrow \dots \rightarrow 1$: são $\log_2 1024 = 10$ divisões.

Exercício D6 ★

Qual sequência está em ordem crescente de crescimento?

- (a) $\log n < n < n \log n < n^2 < 2^n$
- (b) $n < \log n < n^2 < n \log n < 2^n$
- (c) $\log n < n^2 < n < 2^n < n \log n$
- (d) $2^n < n^2 < n \log n < n < \log n$

RESOLUÇÃO (a). É a hierarquia do capítulo 10.

Exercício D7 ★

Numa Fila encadeada com ponteiros para o início e para o fim, qual operação é $O(1)$? (a) enfileirar (b) buscar um valor (c) obter o elemento da posição i (d) percorrer

RESOLUÇÃO (a). Graças ao ponteiro `fin`. As outras precisam percorrer os nós: $O(n)$.

Exercício D8 ★★

A afirmação $f(n) = O(g(n))$ significa que:

- (a) f cresce exatamente como g ;
- (b) f não cresce mais rápido que g , a menos de uma constante, a partir de algum n_0 ;
- (c) f é o pior caso do algoritmo;
- (d) $f(n) \geq g(n)$ para todo n .

RESOLUÇÃO (b). (a) descreve Θ ; (c) confunde notação com caso (capítulo 8); (d) lembra Ω , mas sem a constante e sem o n_0 .



15.5. Bloco E — programação

Exercício E1 ★★

Escreva uma busca binária que conte as comparações e verifique, para vetores ordenados de tamanhos 1.000, 1.000.000 e 1.000.000.000 (use `range`, que não ocupa memória), que o número de comparações no pior caso fica perto de $\log_2 n$.

RESOLUÇÃO

```
import math

def busca_binaria(v, alvo):
    inicio, fim, comparacoes = 0, len(v) - 1, 0
    while inicio <= fim:
        meio = (inicio + fim) // 2
        comparacoes += 1
        if v[meio] == alvo:
            return meio, comparacoes
        if v[meio] < alvo:
            inicio = meio + 1      # descarta a metade da esquerda
        else:
            fim = meio - 1        # descarta a metade da direita
    return -1, comparacoes

for n in (1_000, 1_000_000, 1_000_000_000):
    _, c = busca_binaria(range(n), -1)      # valor ausente: pior caso
    print(n, c, round(math.log2(n), 1))
```

Saída: 9, 19 e 29 comparações, para $\log_2 n$ igual a 10,0, 19,9 e 29,9. Com o valor ausente (menor que todos), a busca faz $\lfloor \log_2 n \rfloor$ comparações — a parte inteira de $\log_2 n$. Um bilhão de elementos, 29 comparações: esse é o poder do $O(\log n)$.

Exercício E2 ★★

Descubra experimentalmente a família de um algoritmo: meça as comparações do Insertion Sort em ordem inversa para n e $2n$ e calcule a razão. Faça o mesmo para a busca linear. O que as razões indicam?

RESOLUÇÃO

```
import sys
sys.path.append("python")      # pasta aula04-complexidade/python
from insertion_sort import comparacoes_invertido

for n in (500, 1000, 2000):
    razao = comparacoes_invertido(2 * n) / comparacoes_invertido(n)
    print(n, round(razao, 2))    # perto de 4: quadrático
```

As razões ficam perto de 4 (para n grande, $\frac{(2n)^2}{n^2} = 4$): crescimento quadrático. Na busca linear, o pior caso tem n comparações, e a razão é 2: crescimento linear. Em geral, se dobrar n multiplica o custo por 2^k , o algoritmo é $\Theta(n^k)$ — a mesma ideia do capítulo 1.

16

Resumo para a prova

NA AULA: slides 49 | NO REPOSITÓRIO: aula04-complexidade/

Tudo o que é essencial nesta apostila, em poucas páginas. Use para revisar antes da prova automática do Grau 1 no LEX — e volte ao capítulo indicado sempre que algo não estiver claro.

16.1. As ideias centrais

1. Medimos algoritmos pelo **número de passos** em função do tamanho da entrada, $T(n)$, e não em segundos (capítulos 1 e 2).
2. O que importa é a **forma do crescimento** para n grande: ficamos com o termo dominante e desprezamos constantes (capítulo 4).
3. O é um teto, Ω é um piso e Θ é o crescimento exato (capítulos 5 a 7).
4. Melhor, pior e caso médio são **situações dos dados**; cada uma é descrita com as notações (capítulos 8 e 9).
5. Trocar de computador multiplica a velocidade por uma constante; trocar de algoritmo pode mudar a família (capítulo 10).

16.2. Fórmulas de bolso

FÓRMULA	PARA QUE SERVE
$1 + 2 + \dots + n = \frac{n(n+1)}{2}$	laços triangulares, deslocamentos, Insertion Sort
$0 + 1 + \dots + (n-1) = \frac{n(n-1)}{2}$	a mesma soma, começando em 0
$\log_2 n = x \iff 2^x = n$	quantas vezes dá para dividir ao meio
$\log_2 n \approx 3,32 \cdot \log_{10} n$	a base do log não importa em O
$n! = n \times (n-1) \times \dots \times 1$	número de ordens de n itens
2^n	número de subconjuntos de n itens
$\frac{d}{dn} n^p = p \cdot n^{p-1}$	derivada: taxa de crescimento (capítulo 11)



16.3. As notações

NOTAÇÃO	IDEIA	DEFINIÇÃO	EXEMPLO
$O(g)$	teto	$f(n) \leq c \cdot g(n)$ para $n \geq n_0$	$3n + 4 = O(n)$
$\Omega(g)$	piso	$f(n) \geq c \cdot g(n)$ para $n \geq n_0$	$3n + 4 = \Omega(n)$
$\Theta(g)$	exato	$c_1 g(n) \leq f(n) \leq c_2 g(n)$ para $n \geq n_0$	$3n + 4 = \Theta(n)$

16.4. A hierarquia e as regras práticas

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$$

PADRÃO DE CÓDIGO	CUSTO
comandos simples	$\Theta(1)$
um laço até n	$\Theta(n)$
dois laços aninhados até n (ou triangulares)	$\Theta(n^2)$
laço que divide ou multiplica por 2	$\Theta(\log n)$
blocos em sequência	soma: fica o maior
laço dentro de laço	multiplica

16.5. Casos que caem na prova

ALGORITMO	MELHOR CASO	CASO MÉDIO	PIOR CASO
busca linear	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
busca binária (vetor ordenado)	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
busca por saltos (vetor ordenado)	$\Theta(1)$	$\Theta(\sqrt{n})$	$\Theta(\sqrt{n})$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Bubble Sort com parada antecipada	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$

E os custos das estruturas das Aulas 1 a 3 estão na tabela do capítulo 13.

ATENÇÃO As pegadinhas mais comuns: (1) achar que O significa “pior caso”; (2) esquecer que x `in lista` é uma busca linear; (3) achar que dividir por 2 muda a família ($\frac{n^2}{4}$ continua $\Theta(n^2)$); (4) comparar algoritmos só com n pequeno; (5) esquecer o “+1” do teste final de um laço ao contar passos.



16.6. Glossário

Algoritmo: sequência finita de passos que resolve um problema.

Análise assintótica: estudo do comportamento do custo quando n cresce muito.

Caso médio: custo esperado para dados típicos (em média).

Custo marginal: quanto o custo aumenta com um elemento a mais; aproximado pela derivada.

Derivada: taxa de crescimento de uma função num ponto; a inclinação da tangente.

Exponencial: função com a variável no expoente, como 2^n ; dobra a cada $+1$.

Fatorial: $n! = n \times (n - 1) \times \dots \times 1$; conta as ordens de n itens.

Função crescente: função cuja saída nunca diminui quando a entrada aumenta.

Inversão: par de elementos fora de ordem num vetor.

Limite: valor do qual uma expressão se aproxima quando n cresce sem parar.

Logaritmo: $\log_2 n$ é quantas vezes n pode ser dividido ao meio até chegar a 1.

Melhor caso: entrada que produz o menor custo possível para um dado n .

Pior caso: entrada que produz o maior custo possível para um dado n ; é uma garantia.

Somatório: soma de uma sequência de termos, escrita com Σ .

Termo dominante: o termo de uma soma que cresce mais rápido e decide o comportamento.

16.7. Símbolos e como ler

SÍMBOLO	LÊ-SE	SIGNIFICADO
O	ó grande	teto do crescimento
Ω	ômega grande	piso do crescimento
Θ	teta grande	crescimento exato
Σ	sigma, somatório	soma de vários termos
n_0	ene zero	a partir de onde a desigualdade vale
$\leq, \geq$	menor ou igual, maior ou igual	comparação
$n \rightarrow \infty$	ene tende ao infinito	n crescendo sem parar
$\log_2 n$	logaritmo de ene na base 2	divisões ao meio até 1
$\sqrt{n}$	raiz quadrada de ene	lado de um quadrado com n casas
$n!$	ene fatorial	produto de 1 até n
$T'(n)$	tê linha de ene	derivada de T
$\lfloor x \rfloor$	piso de x	parte inteira de x (arredonda para baixo)
$\Leftrightarrow$	se, e somente se	as duas afirmações são equivalentes



Mapa: slides da aula e apostila

Use esta tabela para achar, a partir de um slide da aula (ou de um trecho da gravação), onde o assunto é aprofundado nesta apostila.

SLIDES	ASSUNTO	NA APOSTILA	PÁGINA
01 a 03	abertura, objetivos, agenda e como acompanhar	Como usar esta apostila	2
04 e 05	por que analisar; o cronômetro engana	Capítulo 1	4
06 a 08	contando operações; exercício 1	Capítulo 2	8
09 a 16	funções, potências, log, exponencial, fatorial, somatórios	Capítulo 3	13
17 a 19	termo dominante e análise assintótica	Capítulo 4	23
20 a 23	notação O	Capítulo 5	26
24	notação Ω	Capítulo 6	30
25 a 27	notação Θ e resumo	Capítulo 7	32
28	melhor, pior e caso médio	Capítulo 8	35
29 a 36	Insertion Sort	Capítulo 9	38
37 a 40	famílias de funções; tempo e hardware	Capítulo 10	43
41 a 45	derivadas; busca por saltos	Capítulo 11	47
46	regras práticas para analisar código	Capítulo 12	52
47	revisão das estruturas e prova do Grau 1	Capítulos 13 e 16	55
48	atividade prática Campo Minado	Capítulo 14 e enunciado	57
49	encerramento e plano de estudo	Capítulos 15 e 16	65



Materiais da aula no repositório

Todos os arquivos estão em <https://github.com/ProfessorFilipo/EstruturasDados>, na pasta `aula04-complexidade`:

ARQUIVO OU PASTA	CONTEÚDO
<code>python/ e c/</code>	programas das demonstrações (capítulos 2, 9, 10 e 11)
<code>exercicios/campo_minado/</code>	esqueleto da atividade prática, com TODOs
<code>solucoes/campo_minado/</code>	solução de exemplo comentada e <code>GABARITO.md</code>
<code>material/</code>	esta apostila e o enunciado da atividade, em PDF
<code>material/fontes/</code>	fontes em LaTeX (compile com XeLaTeX)